ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

Hệ Cơ Sở Dữ Liệu
(CO2014)

BÁO CÁO BÀI TẬP LỚN

Chủ đề: Hệ thống sàn thương mại điện tử kết hợp Logistics

Giáo viên hướng dẫn	Dương Huỳnh Anh Đức	
Sinh viên thực hiện:	Trần Lý Khánh Vy	2414056
	Lê Nguyễn Nhật Quỳnh	2433199
	Lê Nguyễn Nhật Anh	2410092
	Nguyễn Đức Tiến	2313431
Lớp:	L03 - Nhóm: 6	

TP Hồ Chí Minh, Tháng 5 Năm 2026

Mục lục

I Bài tập lớn 1 **4**

1 Phân tích và mô tả yêu cầu dữ liệu 5

1.1 Tìm hiểu hệ thống liên quan 5

1.2 Mô tả hệ thống đề xuất 7

1.3 Các ràng buộc ngữ nghĩa về các kiểu thực thể, thuộc tính, mối liên kết mà không biểu diễn được bằng (E-)ERD 12

2 Thiết kế (E)-ERD 14

2.1 Các kiểu thực thể và thuộc tính 14

2.2 Các mối quan hệ được biểu thị 18

3 Ánh xạ sang lược đồ CSDL 22

II Bài tập lớn 2 **24**

1 Tạo bảng và dữ liệu mẫu 25

1.1 Tạo bảng dữ liệu và ràng buộc 25

1.1.1 Các nhóm bảng chính trong hệ thống 25

1.1.2 Các loại ràng buộc được áp dụng 25

1.2 Tạo dữ liệu mẫu (Sample Data) 32

1.2.1 Mô tả 32

1.2.2 Input 32

1.2.3 Logic hoạt động 32

1.2.4 Output 33

1.2.5 Demo 33

1.2.6 Code nhập dữ liệu mẫu 34

2 Viết các trigger, thủ tục, hàm 41

2.1 Các thủ tục thêm, xoá, sửa vào bảng Product 41
2.1.1 Thủ tục thêm sản phẩm `sp_insert_product` 41
2.1.2 Thủ tục sửa sản phẩm `sp_update_product` 43
2.1.3 Thủ tục xoá sản phẩm `sp_delete_product` 46
2.2 Triggers 47
2.2.1 Trigger tác nhân 47
2.2.2 Trigger tính thuộc tính dẫn xuất 50
2.3 Viết các thủ tục 55
2.3.1 Thủ tục hiển thị danh sách sản phẩm `sp_get_all_products` . . 56
2.3.2 Thủ tục thống kê doanh thu cửa hàng `sp_get_store_revenue_stats` 58
2.4 Các hàm (Functions) 60
2.4.1 Hàm gợi ý sản phẩm (fn_recommend_products) 60
2.4.2 Hàm phân hạng khách hàng (fn_get_customer_tier) 63
3 Hiện thực ứng dụng 67
3.1 Kiến trúc kết nối ứng dụng với CSDL 67
3.1.1 Kết nối database 67
3.1.2 Cấu hình server và route 68
3.1.3 Định dạng dữ liệu trả về 69
3.2 Giao diện thêm, sửa, xoá dữ liệu vào bảng PRODUCT 70
3.2.1 Thêm sản phẩm 70
3.2.2 Sửa sản phẩm 72
3.2.3 Xoá sản phẩm 73
3.3 Giao diện danh sách, tìm kiếm và lọc sản phẩm 75
3.3.1 Gọi thủ tục `sp_get_all_products` từ giao diện 75
3.3.2 Hiển thị chi tiết và gợi ý mua kèm 76
3.4 Giao diện minh họa hàm và thủ tục khác 78
3.4.1 Giao diện phân hạng khách hàng 78
3.4.2 Giao diện thống kê doanh thu cửa hàng 80
3.5 Đánh giá mức độ đáp ứng yêu cầu 81
3.6 Source code 82

Phân chia công việc - Bài tập lớn 1

STT	Họ và tên	MSSV	Nhiệm vụ	Tỉ lệ hoàn thành
1	Trần Lý Khánh Vy	2414056	Tìm hiểu Shopee, mô tả hệ thống, thiết kế (E-)ERD, viết báo cáo	100%
2	Lê Nguyễn Nhật Quỳnh	2433199	Tìm hiểu Shopee, mô tả hệ thống, thiết kế (E-)ERD	100%
3	Lê Nguyễn Nhật Anh	2410092	Tìm hiểu Shopee, mô tả hệ thống, thiết kế (E-)ERD	100%
4	Nguyễn Đức Tiến	2313431	Tìm hiểu Shopee, ánh xạ lược đồ CSDL	100%

Phân chia công việc - Bài tập lớn 2

STT	Họ và tên	MSSV	Nhiệm vụ	Tỉ lệ hoàn thành
1	Trần Lý Khánh Vy	2414056	Viết SQL các hàm, hoàn thiện Frontend	100%
2	Lê Nguyễn Nhật Quỳnh	2433199	Tạo bảng và dữ liệu mẫu, viết SQL Trigger	100%
3	Lê Nguyễn Nhật Anh	2410092	Viết SQL thủ tục thêm/ sửa/ xoá/ truy vấn, hoàn thiện báo cáo	100%
4	Nguyễn Đức Tiến	2313431	Viết SQL Trigger, hoàn thiện Backend	100%

Part I

Bài tập lớn 1

1 Phân tích và mô tả yêu cầu dữ liệu

1.1 Tìm hiểu hệ thống liên quan

Tên ứng dụng tham khảo: Shopee.

Đường dẫn: https://shopee.vn/

Mô tả tổng quan: Shopee là nền tảng thương mại điện tử hàng đầu Đông Nam Á và Đài Loan ra mắt năm 2015. Hệ thống cho phép người dùng mua bán hàng hóa trực tuyến với trải nghiệm đơn giản, an toàn và nhanh chóng, tích hợp các chức năng như tìm kiếm sản phẩm, đặt hàng, thanh toán và giao nhận. Shopee hướng tới việc kết nối cộng đồng người mua và người bán thông qua công nghệ, đồng thời mang lại trải nghiệm mua sắm trực tuyến thuận tiện và thân thiện cho người dùng.

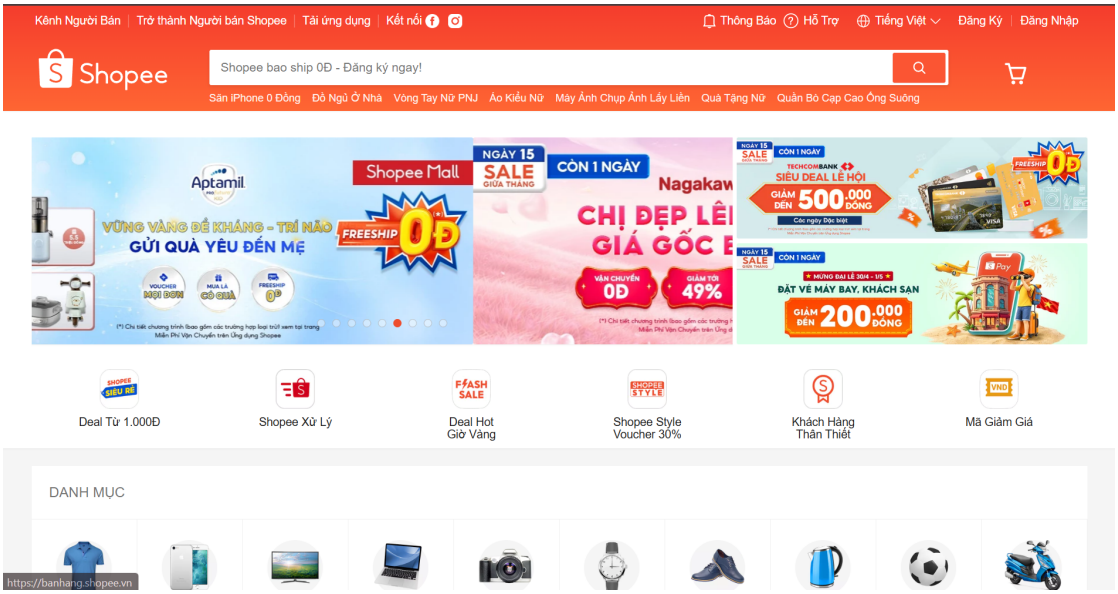
Đối tượng sử dụng chính:

- **Người mua:** Tìm kiếm sản phẩm, đặt hàng, thanh toán và theo dõi đơn hàng.
- **Người bán:** Đăng sản phẩm, quản lý đơn hàng và doanh thu.
- **Quản trị viên:** Kiểm duyệt nội dung, quản lý người dùng và vận hành hệ thống.

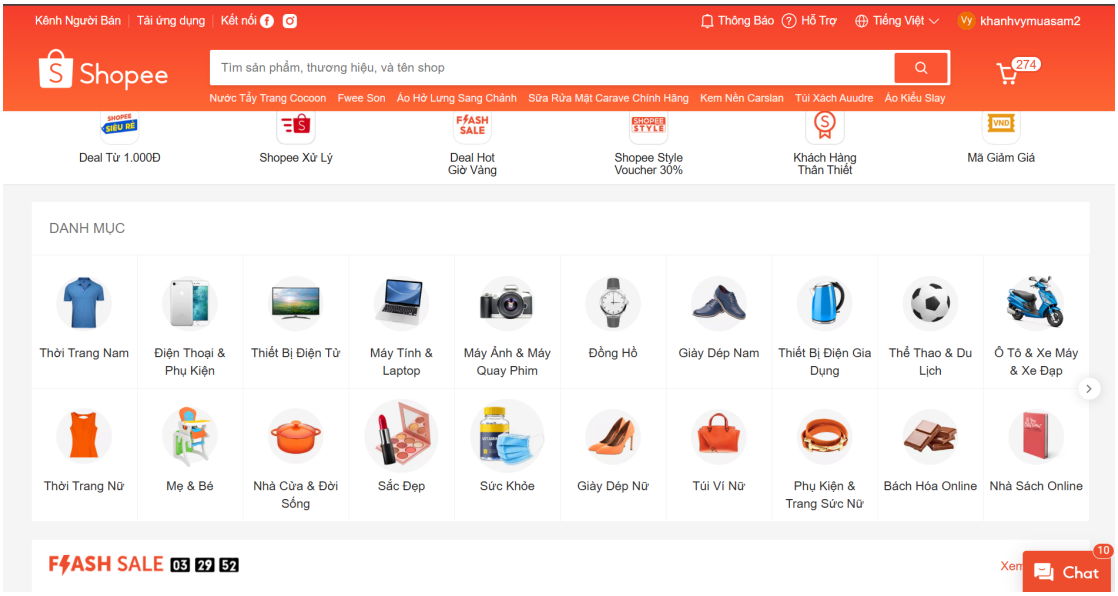
Phân tích nghiệp vụ cơ bản:

- **Xem chi tiết sản phẩm:** Hiển thị thông tin, hình ảnh, giá và đánh giá.
- **Đặt hàng:** Thêm sản phẩm vào giỏ hàng và xác nhận mua.
- **Thanh toán:** Hỗ trợ COD, ví điện tử và thẻ ngân hàng.
- **Theo dõi đơn hàng:** Kiểm tra trạng thái giao hàng và đánh giá sản phẩm.
- **Quản lý sản phẩm:** Người bán đăng và chỉnh sửa sản phẩm, theo dõi đơn hàng.

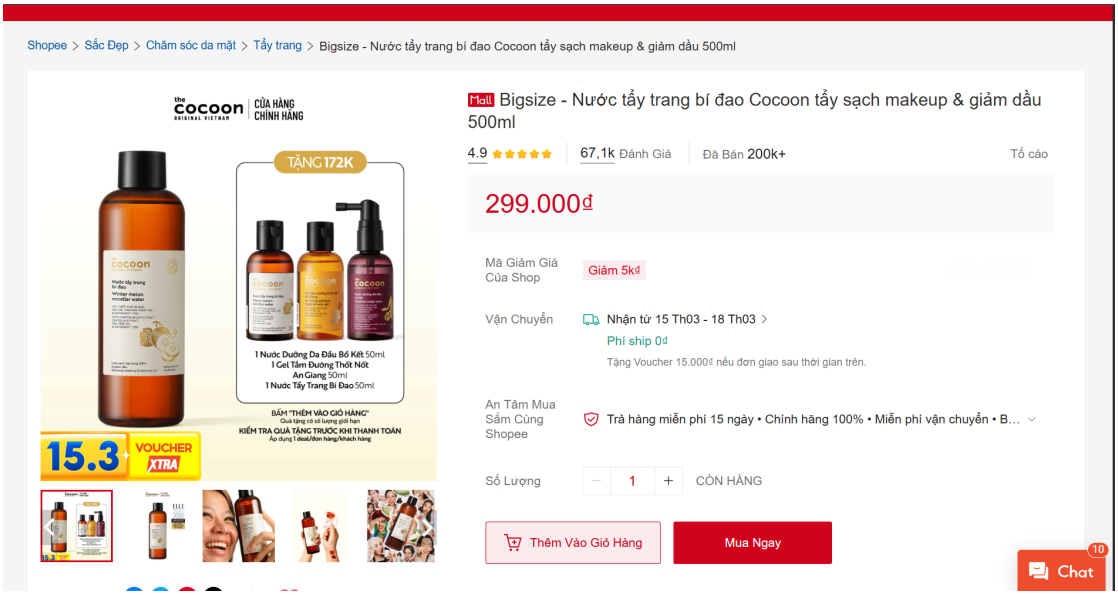
Một số giao diện từ phía người dùng:



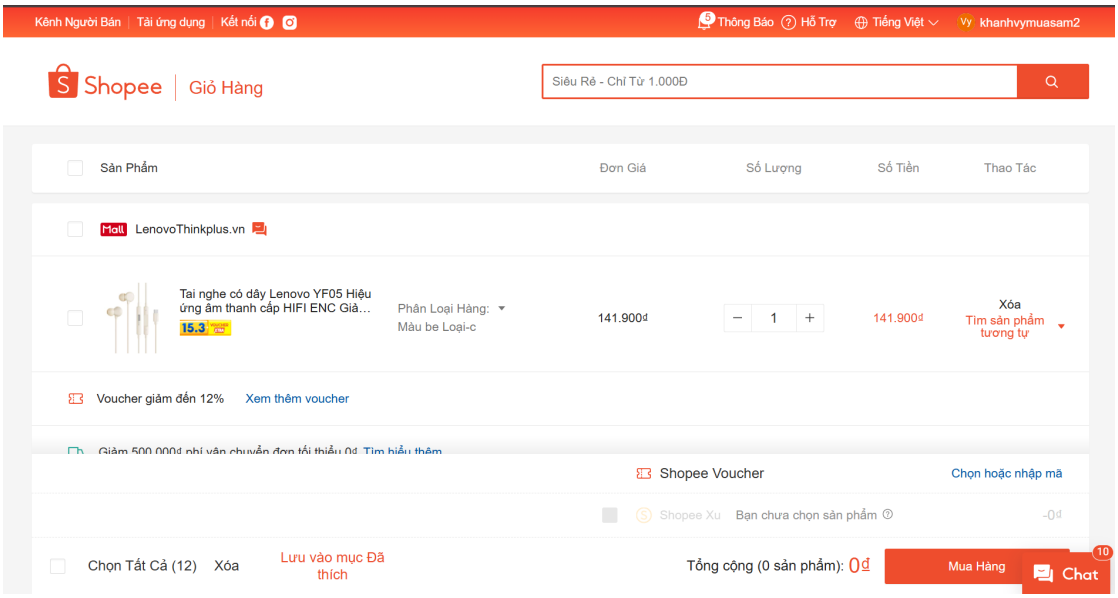
Hình 1.1: Giao diện trang chủ Shopee



Hình 1.2: Danh mục hàng hóa



Hình 1.3: Giao diện chi tiết sản phẩm



Hình 1.4: Giao diện giỏ hàng

1.2 Mô tả hệ thống đề xuất

Nhóm chọn thực hiện hệ thống *OmniLogi* – một nền tảng thương mại điện tử tích hợp mạng lưới hậu cần (*Logistics*) khép kín qua tham khảo mô hình hoạt động của Shopee. Hệ thống này

cho phép người dùng quản lý đồng bộ từ khâu lưu kho, bán hàng, mua sắm, đến vận chuyển và chăm sóc khách hàng. Người dùng trong hệ thống (*USER*) được phân thành 4 nhóm người dùng gồm: Quản trị viên (*ADMIN*), Quản lý cửa hàng (*STORE_MANAGER*), Khách hàng (*CUSTOMER*) và Tài xế (*DRIVER*). Mỗi người dùng phải thuộc một và chỉ một vai trò trong hệ thống. Trong đó, *ADMIN* đóng vai trò là hạt nhân điều hành tối cao, chịu trách nhiệm duy trì sự ổn định và tính minh bạch cho toàn bộ nền tảng. *STORE_MANAGER* đại diện cho các đối tác kinh doanh trực tiếp vận hành các gian hàng trên nền tảng. *CUSTOMER* là đối tượng trung tâm thực hiện các hoạt động mua sắm và tạo ra dòng chảy kinh tế cho nền tảng. *DRIVER* chịu trách nhiệm trực tiếp trong việc luân chuyển hàng hóa từ kho của cửa hàng đến tay người tiêu dùng.

Tất cả tài khoản người dùng *(USER)* đều được lưu trữ các thông tin: mã người dùng (*User_ID*) duy nhất làm khóa chính, cùng các thông tin định danh như tên tài khoản (*User_name*), mật khẩu (*Password*), địa chỉ thư điện tử (*Email*), số điện thoại (*Phone*), giới tính (*Gender*), ngày sinh (*Date_of_birth*), trạng thái hoạt động (*User_status*), ngày tạo (*Created_date*) và tên (*Name*) là một thuộc tính đa trị gồm tên (*First*), tên đệm (*Middle*) và họ (*Last*).

Quản trị viên *(ADMIN)* bổ sung lưu trữ các thông tin đặc thù để phục vụ công tác quản lý gồm: cấp độ quản lý (*Admin_level*) dùng để phân cấp quyền hạn và lần đăng nhập cuối (*Last_login*) nhằm mục đích kiểm soát an ninh và nhật ký hoạt động. Mỗi Quản trị viên có vai trò giám sát những người dùng khác để xử lý các vi phạm điều khoản hoặc hỗ trợ khôi phục tài khoản. Đồng thời, thực hiện giám sát và kiểm duyệt hoạt động của các cửa hàng nhằm đảm bảo tuân thủ các chính sách và tiêu chuẩn vận hành của nền tảng. Ngoài ra, ADMIN còn chịu trách nhiệm quản lý danh mục các sản phẩm của nền tảng. Mỗi ADMIN quản lý nhiều danh mục, mỗi danh mục do một ADMIN phụ trách.

Quản lý cửa hàng *(STORE_MANAGER)* phải bao gồm thông tin mã số thuế (*Tax_code*) làm cơ sở pháp lý để định danh đảm bảo tính minh bạch và trách nhiệm thuế đối với sàn thương mại điện tử. Thông qua quyền sở hữu cửa hàng, Quản lý cửa hàng có toàn quyền kiểm soát các Sản phẩm cửa hàng thuộc phạm vi quản lý của mình. Khi có đơn hàng phát sinh, họ trực tiếp theo dõi các Chi tiết đơn hàng để chuẩn bị hàng hóa và cập nhật trạng thái đơn hàng trước khi bàn giao cho đội ngũ giao vận. Mỗi Quản lý cửa hàng chỉ chịu trách nhiệm quản lý duy nhất một cửa hàng cụ thể và ngược lại.

Trong hệ thống, Khách hàng (CUSTOMER) sở hữu thuộc tính riêng biệt là tuổi (*Age*) – một thuộc tính được tính toán dựa trên ngày sinh để phục vụ việc phân loại phân khúc người dùng và gợi ý sản phẩm phù hợp. Về mặt quản lý mua sắm, mỗi Khách hàng sở hữu duy nhất

một Giỏ hàng (*CART*) để lưu trữ tạm thời các sản phẩm dự định mua. Thực thể *CART* bao gồm: mã định danh giỏ hàng *(Cart_ID)* và trạng thái giỏ hàng *(cart_status)*. Mỗi Khách hàng có quyền thiết lập và sở hữu nhiều Địa chỉ (*ADDRESS*) nhận hàng khác nhau cho phép linh hoạt lựa chọn nơi giao hàng phù hợp cho từng giao dịch cụ thể. Mỗi địa chỉ bao gồm các thông tin: mã định danh địa chỉ duy nhất (*Address_ID*), tên người nhận (*Recipient_name*), số điện thoại người nhận (*Recipient_phone*), thông tin chi tiết gồm số nhà và tên đường (*Details*), tỉnh thành (*Province/City*) và loại địa chỉ (*Address_type*) như nhà riêng hoặc văn phòng. Khách hàng trực tiếp kiểm soát trạng thái của các món hàng từ lúc thanh toán cho đến khi nhận hàng thành công. Sau khi nhận hàng, khách hàng có thể viết các đánh giá cho từng món hàng đã mua. Ngoài ra, Khách hàng còn sở hữu các bản ghi Điểm thưởng (*LOYALTY_POINTS*). Thực thể yếu *LOYALTY_POINTS* bao gồm các thuộc tính: mã định danh cho lần tích/trừ điểm (Transaction_ID), loại biến đổi điểm tích/trừ điểm *(Transaction)*, thời hạn sử dụng điểm *(Expiry_date)*, tổng điểm*(Total_points)*. Mỗi khi thực hiện Đơn hàng khách hàng có thể sử dụng toàn bộ điểm thưởng đang có để khấu trừ giá tiền. Và mỗi khi Đơn hàng thành công thì sẽ nhận được một bản ghi Điểm thưởng vs số điểm thưởng tương ứng với giá trị đơn hàng.

Đối với Tài xế *(DRIVER)*, thực thể này được bổ sung các thuộc tính riêng biệt bao gồm: số bằng lái (*License_num*) để xác minh tư cách pháp lý, loại xe (*Vehicle_type*) để phân phối đơn hàng phù hợp và trạng thái tài xế (*Driver_status*) giúp hệ thống nhận diện tài xế đang "Sẵn sàng" hay "Đang bận". Đặc biệt, thuộc tính *Total_deliveries* là một thuộc tính dẫn xuất, được hệ thống tự động cập nhật dựa trên số lượng vận đơn mà tài xế đã hoàn thành. Về mặt nghiệp vụ, hệ thống giao vận cung cấp nhiều *Phương tiện* (*Vehicle*), cho phép mỗi Tài xế được lựa chọn sử dụng nhiều phương tiện khác nhau và mỗi phương tiện cũng được sử dụng bởi nhiều Tài xế khác nhau. Mỗi phương tiện *(Vehicle)* bao gồm các thông tin: mã định danh phương tiện *(Vehicle_ID)*, biển số *(Plate_number)*, thể loại *(Type)* và trạng thái *(Status)* đang "sẵn sàng" hay "đang bận".

Hệ thống *OmniLogi* tổ chức quản lý hàng hóa và phân loại thông qua Danh mục (*CATEGORY*). Thực thể *CATEGORY* đóng vai trò định hình cấu trúc cây sản phẩm với các thuộc tính mã danh mục (*Category_ID*) duy nhất và tên danh mục (*Category_name*). Đặc điểm nổi bật của thực thể này là mối liên kết đệ quy (*Parent_of*), cho phép thiết lập quan hệ cha-con giữa các danh mục để tạo ra sự phân cấp sâu (như Điện tử > Điện thoại > Smartphone), giúp tối ưu hóa việc tìm kiếm và quản lý. Song song đó, thực thể Sản phẩm (*PRODUCT*) tập trung lưu trữ các thông tin định danh chung trên toàn sàn bao gồm mã sản phẩm (*Product_ID*) làm khóa chính, tên (*Name*), thương hiệu (*Brand*) và mô tả (*Description*) chi tiết về đặc tính kỹ

thuật của sản phẩm. Về mặt vận hành thương mại, hệ thống quản lý các đơn vị bán hàng thông qua các Cửa hàng (*STORE*) với các thuộc tính mã cửa hàng (*Store_ID*) làm khóa chính, tên (*Name*), địa chỉ kinh doanh (*Store_address*) và mô tả (*Description*) gian hàng. Để giải quyết bài toán cùng một sản phẩm nhưng có giá bán và tồn kho khác nhau tại mỗi shop, hệ thống sử dụng thực thể yếu Sản phẩm cửa hàng (*STORE_PRODUCT*) làm cầu nối giữa *STORE* và *PRODUCT*. Thực thể này sử dụng mã sản phẩm cửa hàng (*SProduct_ID*) làm khóa bộ phận và lưu trữ các thông tin kinh doanh đặc thù như giá bán hiện tại (*Price*) cùng số lượng tồn kho (*Stock_quantity*).

Để đảm bảo tính an toàn và hợp pháp cho các sản phẩm được giao dịch, mọi sản phẩm trước khi được đăng bán đều phải trải qua quy trình kiểm duyệt bởi Quản trị viên (ADMIN). Cụ thể, chủ cửa hàng (STORE_MANAGER) sẽ gửi đề xuất thông tin sản phẩm mới đến hệ thống. ADMIN có trách nhiệm kiểm tra, đánh giá tính hợp lệ của sản phẩm (không phải hàng cấm, không vi phạm chính sách, thông tin chính xác). Sau khi sản phẩm được ADMIN phê duyệt, một bản ghi mới được thêm vào thực thể *PRODUCT* với trạng thái *'Approved'*. Đồng thời, ADMIN cũng thiết lập liên kết giữa sản phẩm này với cửa hàng tương ứng thông qua thực thể liên kết *STORE_PRODUCT*, xác định cửa hàng sẽ bán sản phẩm đó.

Mỗi sản phẩm trong hệ thống được quản lý tập trung tại thực thể *PRODUCT* với các thuộc tính: mã sản phẩm (*Product_ID*), tên (*Name*), thương hiệu (*Brand*), mô tả (*Description*), hình ảnh (*image_url*) và trạng thái kiểm duyệt (*status*). Một sản phẩm có thể được bán tại nhiều cửa hàng khác nhau thông qua thực thể liên kết *STORE_PRODUCT* (quan hệ nhiều - nhiều giữa *STORE* và *PRODUCT*), cho phép hệ thống linh hoạt trong việc quản lý cùng một sản phẩm được phân phối bởi nhiều nhà bán lẻ khác nhau, đồng thời hỗ trợ các tính năng thống kê bán hàng theo nhà phân phối sau này.

Để quản lý chi tiết các mặt hàng trong giai đoạn dự định mua, thực thể yếu *CART_ITEM* được thiết lập với mã định danh bộ phận là *CartItem_ID* và thuộc tính *CartItem_quantity* nhằm lưu trữ số lượng sản phẩm mà khách hàng lựa chọn. Thực thể này có mối liên kết định danh với thực thể chủ *CART*. Trong đó một giỏ hàng có thể chứa nhiều chi tiết món hàng và mỗi chi tiết món hàng bắt buộc phải thuộc về duy nhất một giỏ hàng. Đồng thời, mỗi món hàng trong giỏ phải được ánh xạ trực tiếp đến một sản phẩm cụ thể của một cửa hàng nhất định trước khi chuyển đổi thành đơn hàng chính thức.

Tiếp nối quy trình vận hành, khi khách hàng xác nhận giỏ hàng, mọi giao dịch sẽ được quản lý chặt chẽ với sự tích hợp trực tiếp của hệ thống điểm thưởng. Thực thể trung tâm của luồng nghiệp vụ này là Đơn hàng (*ORDER*), nơi lưu trữ các thông tin vận hành quan trọng

như ngày dự kiến giao (*Expected_ date*), ngày thực tế giao (*Actual_ date*), trạng thái đơn hàng (*Order_ status*) và thuộc tính dẫn xuất Tổng tiền (*Total_ price*) được hệ thống tự động tổng hợp dựa trên giá trị các món hàng và trừ đi phần giá trị quy đổi từ điểm tích lũy của khách hàng theo công thức:

$$\text{Total_price} = \sum(\text{Quantity} \times \text{Price_of_purchase}) - \text{Discount_from_Loyalty}$$

Mỗi Đơn hàng *(ORDER)* có thể áp dụng được nhiều bản ghi điểm thưởng, nhưng mỗi bản ghi chỉ áp dụng được cho một đơn hàng. Mỗi đơn hàng thành công có thể quy đổi thành một bản ghi điểm thưởng duy nhất và mỗi bản ghi điểm thưởng thuộc về một đơn hàng thành công duy nhất. Đồng thời, mỗi đơn hàng còn gắn liền với các chi tiết đơn hàng (*ORDER_ ITEM*) nhằm ghi lại danh sách sản phẩm cụ thể trong mỗi giao dịch. Với thuộc tính Giá mua (*Price_ of_ purchase*), hệ thống thực hiện việc lưu trữ giá sản phẩm ngay tại thời điểm chốt đơn để bảo vệ quyền lợi người dùng, tránh các biến động giá từ phía cửa hàng sau khi giao dịch đã khởi tạo. Bên cạnh đó, hệ thống cung cấp tính năng quản lý dòng tiền qua thực thể Thanh toán (*PAYMENT*) với mã thanh toán (*Payment_ ID*) duy nhất, ghi nhận số tiền thực tế khách hàng phải trả sau khi đã khấu trừ điểm thưởng, hỗ trợ nhiều phương thức (*Method*) như COD hay ví điện tử, đồng thời lưu vết trạng thái (*Payment_ status*) và ngày giờ giao dịch (*Payment_ date*).

Sau khi đơn hàng được xác nhận và chuẩn bị, hệ thống *OmniLogi* vận hành mạng lưới giao vận thông qua thực thể Vận đơn (*SHIPMENT*) với mã vận đơn (*Shipment_ ID*) duy nhất và trạng thái (*Status*) cụ thể để điều phối quá trình luân chuyển hàng hóa. Để đảm bảo tính minh bạch tối đa cho cả khách hàng và quản trị viên, thực thể yếu *TRACKING_ HISTORY* được thiết lập thông qua mối liên kết định danh với thực thể SHIPMENT. Mỗi bản ghi nhật ký hành trình được phân biệt bởi mã định danh bộ phận *(Tracking_ ID)*, đại diện cho một mốc di chuyển cụ thể của gói hàng. Khi gói hàng luân chuyển qua các trạm, hệ thống ghi nhận dữ liệu về vị trí địa lý *(Current_ location)* và trạng thái vận hành tại thời điểm đó *(Status_ updated)*. Đồng thời, thuộc tính *Time_ stamp* tự động lưu lại dấu thời gian chính xác của sự kiện, cho phép các bên liên quan truy vết toàn bộ lộ trình di chuyển của vận đơn theo thời gian thực. Một vận đơn *(SHIPMENT)* có thể bao gồm nhiều bản ghi lịch sử hành trình *(TRACKING_ HISTORY)* tương ứng với các mốc thời gian và địa điểm khác nhau trong quá trình vận chuyển. Ngược lại, mỗi bản ghi lịch sử hành trình chỉ thuộc về duy nhất một vận đơn cụ thể.

Bên cạnh luồng hậu cần, hệ thống còn chú trọng xây dựng uy tín sàn thông qua tính năng

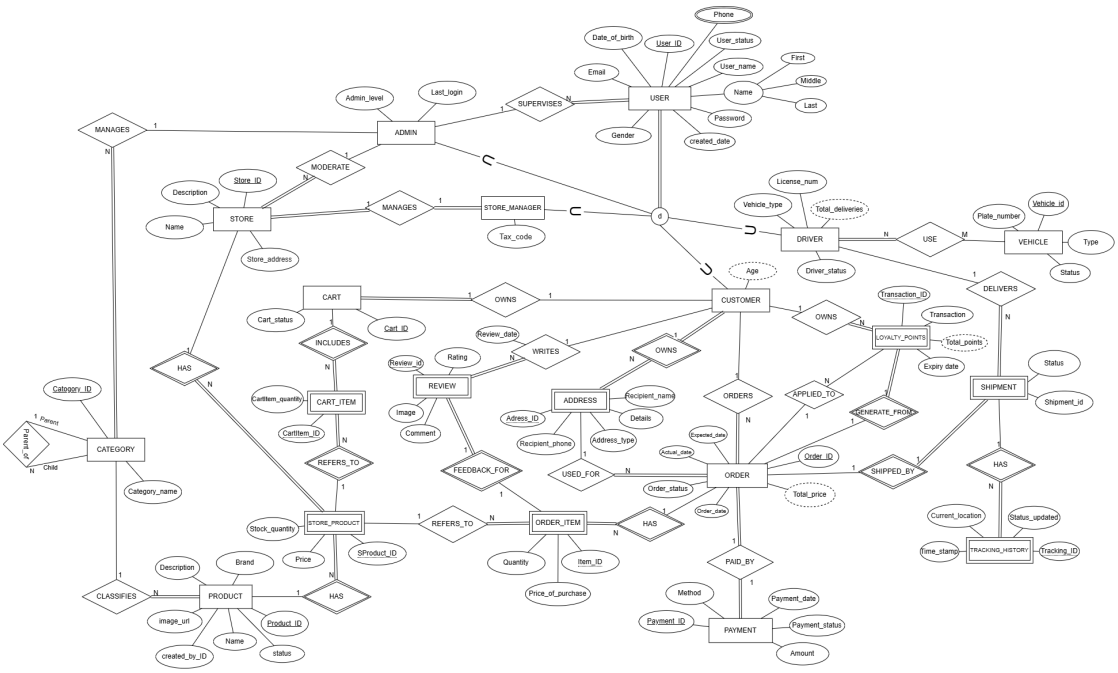
Đánh giá (*REVIEW*) sau khi nhận hàng thành công. Khách hàng có thể phản hồi cho từng sản phẩm cụ thể, lưu trữ các thông tin về điểm xếp hạng (*Rating*), nhận xét chi tiết (*Comment*), hình ảnh thực tế (*Image*) và ngày thực hiện đánh giá (*Review_ date*). Việc từng Đánh giá phụ thuộc trực tiếp vào từng chi tiết món hàng thay vì toàn bộ đơn hàng giúp *OmniLogi* đảm bảo tính khách quan và độ tin cậy của thông tin, hỗ trợ những khách hàng sau có cái nhìn chính xác nhất về chất lượng sản phẩm từ từng cửa hàng cụ thể.

1.3 Các ràng buộc ngữ nghĩa về các kiểu thực thể, thuộc tính, mối liên kết mà không biểu diễn được bằng (E-)ERD

1. Mỗi tài khoản người dùng trong hệ thống phải có một địa chỉ email duy nhất; tuyệt đối không cho phép hai người dùng khác nhau đăng ký cùng một địa chỉ email.
2. Quản trị viên *(ADMIN)* có cấp độ quản lý thấp hơn sẽ chịu sự giám sát từ người dùng có cấp độ quản lý cao hơn dựa trên giá trị của thuộc tính *Admin_ level*.
3. Số lượng sản phẩm khách hàng đặt mua không được vượt quá số lượng tồn kho của sản phẩm tại cửa hàng tại thời điểm đặt hàng.
4. Một đơn hàng chỉ được phép thanh toán thành công một lần duy nhất; sau khi giao dịch thành công, hệ thống không cho phép thực hiện thêm giao dịch thanh toán nào khác cho đơn hàng đó.
5. Mỗi giao dịch thanh toán phải ghi nhận rõ một phương thức thanh toán hợp lệ duy nhất có trong danh mục hỗ trợ của hệ thống.
6. Cơ chế quy đổi được quy định theo tỷ lệ mỗi điểm tương ứng với 1 VND.
7. Giá trị quy đổi từ điểm thưởng không được vượt quá 50% tổng giá trị hàng hóa của đơn hàng đó.
8. Phương tiện *Vehicle* chỉ được sử dụng khi ở trạng thái "sẵn sàng".
9. Tại cùng một thời điểm, một tài xế giao hàng không thể nhận nhiều hơn 20 đơn hàng đang ở trạng thái "đang giao".
10. Khách hàng chỉ có thể viết đánh giá cho sản phẩm khi đơn hàng tương ứng đã được cập nhật sang trạng thái "Đã giao hàng thành công".
11. Dấu thời gian (*Time_ stamp*) của các bản ghi trong lịch sử hành trình phải tuân theo trình tự thời gian; bản ghi sau phải lớn hơn bản ghi trước đối với cùng một vận đơn.

12. Điểm xếp hạng (*Rating*) trong các đánh giá phải nằm trong phạm vi từ 1 đến 5 sao.

2 Thiết kế (E)-ERD



Hình 2.1: Hình vẽ (E)-ERD

Link Draw.io chứa (E-)ERD có độ phân giải cao: Click here

2.1 Các kiểu thực thể và thuộc tính

Hệ thống được thiết kế bao gồm 13 thực thể mạnh và 7 thực thể yếu.

USER (Thực thể mạnh - Cha):

- User_ID (Khóa chính)
- Name (Thuộc tính hợp thành): First, Middle, Last
- User_name
- Password
- Email
- Phone

- Gender
- Date_of_birth
- User_status
- Created_date

ADMIN (Thực thể mạnh - Con của USER):

- Admin_level
- Last_login

STORE_MANAGER (Thực thể mạnh - Con của USER):

- Tax_code

DRIVER (Thực thể mạnh - Con của USER):

- License_num
- Vehicle_type
- Driver_status
- Total_deliveries (Thuộc tính dẫn xuất từ số lượng bản ghi SHIPMENT có trạng thái thành công)

CUSTOMER (Thực thể mạnh - Con của USER):

- Age (Thuộc tính dẫn xuất từ Date_of_birth của thực thể USER)

CATEGORY (Thực thể mạnh):

- Category_ID (Khóa chính)
- Category_name

STORE (Thực thể mạnh):

- Store_ID (Khóa chính)
- Name
- Store_address

- Description

PRODUCT (Thực thể mạnh):

- Product_ID (Khóa chính)
- Name
- Brand
- Description
- Image_url
- Status (Giá trị: 'Pending', 'Approved', 'Rejected')

CART (Thực thể mạnh):

- Cart_ID (Khóa chính)
- Status_updated

ORDER (Thực thể mạnh):

- Order_ID (Khóa chính)
- Order_status
- Order_date
- Expected_date
- Actual_date
- Total_price (Thuộc tính dẫn xuất từ thuộc tính Price_of_purchase của thực thể ORDER_ITEM)

PAYMENT (Thực thể mạnh):

- Payment_ID (Khóa chính)
- Method
- Amount
- Payment_status

- Payment_date

SHIPMENT (Thực thể mạnh):

- Shipment_id (Khóa chính)
- Status

VEHICLE (Thực thể mạnh):

- Vehicle_id (Khóa chính)
- Plate_number
- Type

LOYALTY_POINTS (Thực thể yếu - Có thực thể chủ là ORDER):

- Transaction_ID (Khóa bộ phận)
- Transaction
- Total_Points
- Expiry_date

STORE_PRODUCT (Thực thể yếu - Có thực thể chủ là STORE và PRODUCT):

- SProduct_ID (Khóa bộ phận)
- Price
- Stock_quantity
- Categorical (Thuộc tính đa trị)

CART_ITEM (Thực thể yếu - Có thực thể chủ là CART và STORE_PRODUCT):

- CartItem_ID (Khóa bộ phận)
- CartItem_quantity

ADDRESS (Thực thể yếu - Có thực thể chủ là CUSTOMER):

- Address_ID (Khóa bộ phận)

- Recipient_name
- Recipient_phone
- Details
- Address_type

REVIEW (Thực thể yếu - Có thực thể chủ là ORDER_ITEM):

- Review_id (Khóa bộ phận)
- Rating
- Comment
- Image

ORDER_ITEM (Thực thể yếu - Có thực thể chủ là ORDER):

- Item_ID (Khóa bộ phận)
- Quantity
- Price_of_purchase

TRACKING_HISTORY (Thực thể yếu - Có thực thể chủ là SHIPMENT):

- Tracking_ID (Khóa bộ phận)
- Current_location
- Time_stamp
- Status_updated

2.2 Các mối quan hệ được biểu thị

Hệ thống gồm 27 mối quan hệ:

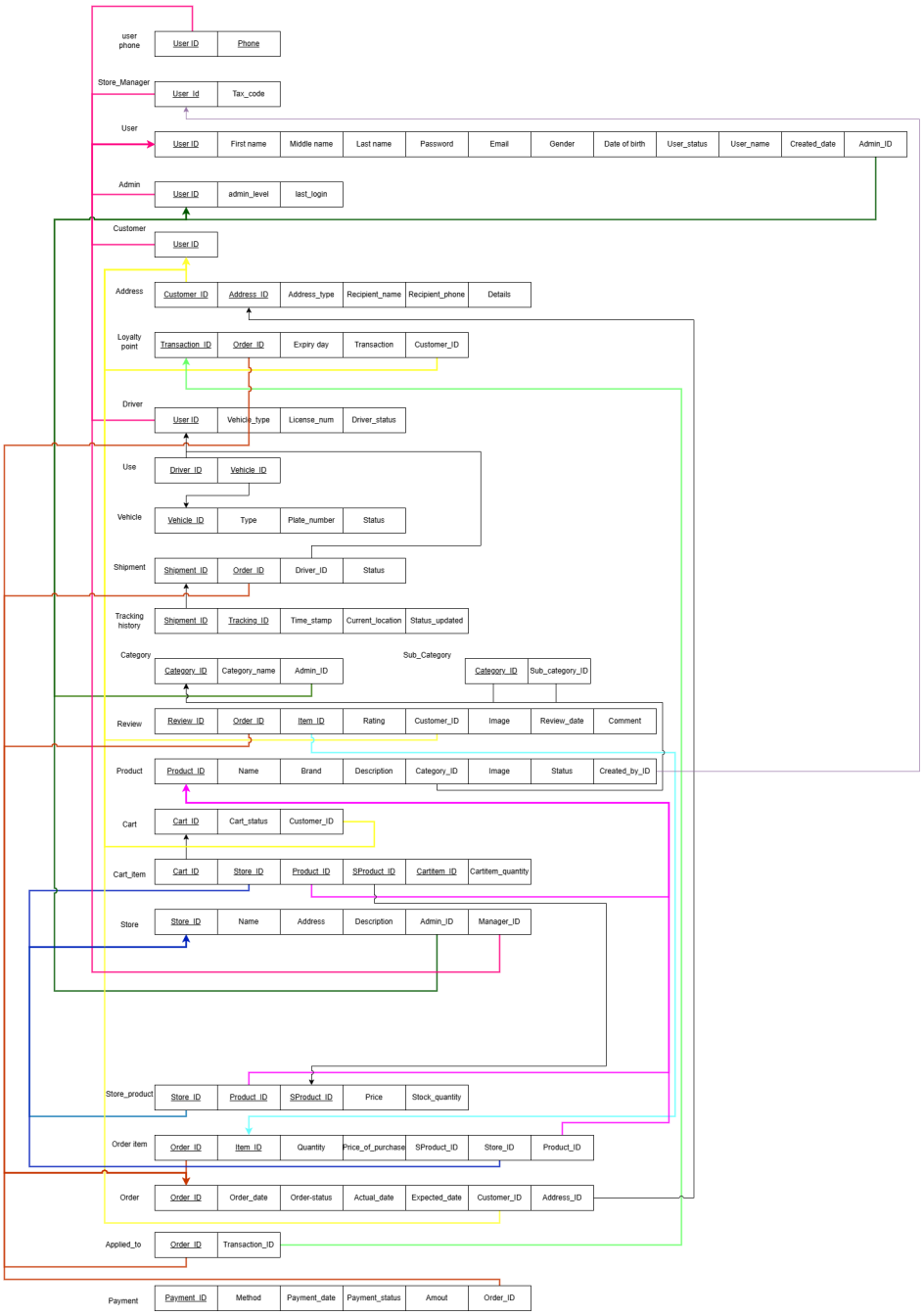
- **MANAGES:** STORE_MANAGER (1) – STORE (1). Mỗi Quản lý cửa hàng trực tiếp điều hành duy nhất một cửa hàng, và mỗi cửa hàng bắt buộc phải có một Quản lý chịu trách nhiệm pháp lý.
- **OWNS:** CUSTOMER (1) – CART (1). Mỗi khách hàng sở hữu duy nhất một giỏ hàng và mỗi giỏ hàng bắt buộc thuộc về một khách hàng duy nhất.

- **FEEDBACK_FOR:** REVIEW (1) – ORDER_ITEM (1). Mỗi đánh giá phản hồi cho duy nhất một món hàng đã mua, mỗi món hàng đã được đặt có thể nhận tối đa một đánh giá từ khách hàng.
- **PAID_BY:** ORDER (1) – PAYMENT (1). Mỗi đơn hàng gắn liền với một giao dịch thanh toán duy nhất và mỗi thanh toán bắt buộc phải thuộc về một đơn hàng.
- **SHIPPED_BY:** ORDER (1) – SHIPMENT (1). Mỗi đơn hàng sau khi xác nhận sẽ khởi tạo duy nhất một vận đơn vận chuyển tương ứng. Mỗi mã vận đơn tương ứng với duy nhất một đơn hàng cụ thể.
- **APPLIED_TO:** LOYALTY_POINTS (N) – ORDER (1). Một đơn hàng có thể áp dụng đồng thời nhiều bản ghi điểm thưởng khác nhau để khấu trừ vào giá trị thanh toán. Tổng giá trị quy đổi sẽ được cộng dồn từ các bản ghi điểm được chọn, trong khi mỗi bản ghi điểm chỉ được phép sử dụng duy nhất cho một đơn hàng để đảm bảo tính minh bạch trong đối soát.
- **GENERATE_FROM:** ORDER (1) – LOYALTY_POINTS(1). Một đơn hàng sau khi hoàn tất chỉ sinh ra duy nhất một bản ghi tích điểm. Một bản ghi tích điểm phải được truy xuất nguồn gốc từ duy nhất một đơn hàng.
- **USED_FOR:** ADDRESS (1) – ORDER (N). Một bản ghi địa chỉ đã lưu có thể được khách hàng sử dụng cho nhiều đơn hàng khác nhau. Ngược lại, tại một thời điểm thanh toán, mỗi đơn hàng chỉ được phép liên kết với duy nhất một địa chỉ giao hàng cụ thể để xác định điểm đến cho vận đơn.
- **SUPERVISES:** ADMIN (1) – USER (N). Mỗi Quản trị viên có thể giám sát hoạt động của nhiều tài khoản trên hệ thống, mỗi tài khoản khác buộc chịu sự giám sát từ một Quản trị viên.
- **MODERATES:** ADMIN (1) – STORE (N). Mỗi ADMIN có quyền giám sát và kiểm duyệt mọi hoạt động của nhiều STORE. Mỗi STORE bắt buộc chịu sự giám sát của duy nhất một ADMIN.
- **Parent_of:** CATEGORY (1) – CATEGORY (N). Một danh mục có thể đóng vai trò là danh mục cha của nhiều danh mục con khác nhằm tạo cấu trúc phân cấp đệ quy. Mỗi danh mục con phải là con từ một danh mục cha.

- **CLASSIFIES:** CATEGORY (1) – PRODUCT (N). Mỗi danh mục có thể phân loại cho nhiều sản phẩm khác nhau, mỗi sản phẩm bắt buộc phải thuộc về một danh mục cụ thể.
- **HAS:** PRODUCT (1) – STORE_PRODUCT (N). Một sản phẩm gốc có thể có nhiều phiên bản bán lẻ tại các cửa hàng khác nhau với giá và tồn kho riêng biệt. Mỗi sản phẩm cụ thể thì chỉ thuộc một sản phẩm gốc duy nhất trong danh mục.
- **HAS:** STORE (1) – STORE_PRODUCT (N). Mỗi cửa hàng sở hữu nhiều sản phẩm cụ thể trong kho hàng của mình, các sản phẩm này bắt buộc phải thuộc về một cửa hàng.
- **INCLUDES:** CART (1) – CART_ITEM (N). Mỗi giỏ hàng có thể chứa nhiều món hàng khác nhau, các món hàng này bắt buộc phải nằm trong một giỏ hàng cụ thể.
- **REFERS_TO:** CART_ITEM (N) – STORE_PRODUCT (1). Nhiều món hàng trong giỏ có thể tham chiếu đến thông tin của cùng một sản phẩm tại một cửa hàng nhất định. Mỗi sản phẩm cụ thể tại cửa hàng tham chiếu tới duy nhất một món hàng trong giỏ hàng.
- **OWNS:** CUSTOMER (1) – ADDRESS (N). Mỗi khách hàng có thể lưu trữ nhiều địa chỉ nhận hàng, mỗi địa chỉ này bắt buộc phải gắn liền với tài khoản của khách hàng đó.
- **ORDERS:** CUSTOMER (1) – ORDER (N). Mỗi khách hàng có thể thực hiện nhiều đơn hàng theo thời gian, mỗi đơn hàng bắt buộc phải được đặt bởi một khách hàng.
- **HAS:** ORDER (1) – ORDER_ITEM (N). Mỗi đơn hàng bắt buộc bao gồm ít nhất một chi tiết món hàng, mỗi chi tiết món hàng bắt buộc thuộc về duy nhất một đơn hàng.
- **REFERS_TO:** ORDER_ITEM (N) – STORE_PRODUCT (1). Mỗi chi tiết món hàng trong đơn phải liên kết với một sản phẩm thực tế trong kho để xác định giá tại thời điểm mua. Một sản phẩm cụ thể có thể là chi tiết trong nhiều đơn hàng.
- **DELIVERS:** DRIVER (1) – SHIPMENT (N). Mỗi tài xế có thể đảm nhận giao nhiều vận đơn, mỗi vận đơn tại một thời điểm bắt buộc được giao bởi một tài xế cụ thể.
- **HAS:** SHIPMENT (1) – TRACKING_HISTORY (N). Mỗi vận đơn có một lịch sử ghi lại lộ trình di chuyển, lịch sử này bắt buộc thuộc về một vận đơn duy nhất.
- **WRITES:** CUSTOMER (1) – REVIEW (N). Một Khách hàng có quyền thực hiện nhiều bài đánh giá cho các sản phẩm khác nhau mà họ đã trải nghiệm. Ngược lại, mỗi bản ghi Đánh giá cụ thể phải được khởi tạo và sở hữu bởi duy nhất một Khách hàng.

- **OWNS:** CUSTOMER (1) – LOYALTY_POINTS (N). Mỗi khách hàng sở hữu nhiều bản ghi Điểm thưởng, mỗi bản ghi Điểm thưởng thuộc về một khách hàng.
- **HAS:** PRODUCT (1) – STORE_PRODUCT (N). Mỗi sản phẩm gốc trong hệ thống có thể được phân phối tại nhiều cửa hàng khác nhau dưới dạng sản phẩm cửa hàng. Mỗi sản phẩm cửa hàng bắt buộc phải tham chiếu đến một sản phẩm gốc duy nhất.
- **HAS:** ORDER (1) – ORDER_ITEM (N). Mỗi đơn hàng có thể bao gồm một hoặc nhiều chi tiết món hàng khác nhau. Mỗi chi tiết món hàng bắt buộc phải thuộc về một đơn hàng cụ thể và không thể tồn tại độc lập, nhằm lưu vết giá và số lượng tại thời điểm khách hàng chốt đơn.
- **USE:** DRIVER (N) – VEHICLE (M). Mỗi tài xế có thể lựa chọn sử dụng nhiều loại xe, mỗi loại xe có thể được sử dụng bởi nhiều tài xế.

3 Ánh xạ sang lược đồ CSDL



Hình 3.1: Hình vẽ ánh xạ

Link Draw.io chứa lược đồ có độ phân giải cao: Click here

Part II

Bài tập lớn 2

1 Tạo bảng và dữ liệu mẫu

1.1 Tạo bảng dữ liệu và ràng buộc

Phần này thực hiện việc tạo toàn bộ cơ sở dữ liệu `Omnilogi` – một hệ thống sàn thương mại điện tử tích hợp logistics. Hệ thống được thiết kế với 25 bảng dữ liệu, bao gồm đầy đủ các ràng buộc về khóa chính, khóa ngoại, ràng buộc dữ liệu (`CHECK`, `DEFAULT`, `UNIQUE`) và các ràng buộc tham chiếu (`ON DELETE CASCADE / SET NULL / RESTRICT`).

1.1.1 Các nhóm bảng chính trong hệ thống

Hệ thống được chia thành 7 nhóm bảng chính:

- **Nhóm người dùng và phân quyền:** `USER`, `USER_PHONE`, `ADMIN`, `STORE_MANAGER`, `CUSTOMER`, `DRIVER`
- **Nhóm phương tiện vận chuyển:** `VEHICLE`, `USE`
- **Nhóm danh mục:** `CATEGORY`, `SUB_CATEGORY`
- **Nhóm cửa hàng và sản phẩm:** `PRODUCT`, `STORE`, `STORE_CATEGORICAL`, `STORE_PRODUCT`
- **Nhóm giỏ hàng và đơn hàng:** `CART`, `CART_ITEM`, `ORDER`, `ORDER_ITEM`
- **Nhóm thanh toán, tích điểm và vận chuyển:** `PAYMENT`, `LOYALTY_POINTS`, `APPLIED_TO`, `SHIPMENT`, `TRACKING_HISTORY`
- **Nhóm địa chỉ và đánh giá:** `ADDRESS`, `REVIEW`

1.1.2 Các loại ràng buộc được áp dụng

1.1.2.1 Ràng buộc khóa chính (Primary Key)

- Tất cả 25 bảng đều có khóa chính được định nghĩa rõ ràng.
- Các bảng có khóa chính đơn giản: `USER(User_ID)`, `VEHICLE(Vehicle_id)`, `CATEGORY(Category_ID)`, `STORE(Store_ID)`, `PRODUCT(Product_ID)`, `STORE_PRODUCT(SProduct_ID)`, `CART(Cart_ID)`, `CART_ITEM(CartItem_ID)`, `ORDER(Order_ID)`, `PAYMENT(Payment_ID)`, `SHIPMENT(Shipment_ID)`, `REVIEW(Review_ID)`. Trong đó, các bảng này sử dụng `AUTO_INCREMENT` (MySQL) để tự động tăng khóa chính.
- Các bảng có khóa chính phức hợp: `USER_PHONE (User_ID, Phone)`, `SUB_CATEGORY (Category_ID, Sub_category_ID)`, `STORE_CATEGORICAL (Store_ID, Categorical)`, `ORDER_ITEM (Item_ID,`

Order_ID), LOYALTY_POINTS (Transaction_ID, Order_ID), TRACKING_HISTORY (Tracking_ID, Shipment_ID), ADDRESS (Address_ID, Customer_ID), USE (User_ID, Vehicle_ID), APPLIED_TO (Order_ID) (khóa chính là Order_ID, nhưng đồng thời là khóa ngoại).

1.1.2.2 Ràng buộc khóa ngoại (Foreign Key)

- Hệ thống sử dụng tổng cộng 39 ràng buộc khóa ngoại nhằm đảm bảo tính toàn vẹn tham chiếu giữa các bảng.
- Các quan hệ phụ thuộc mạnh (weak entities) như USER_PHONE, ADDRESS, ORDER_ITEM sử dụng ON DELETE CASCADE để tự động xóa dữ liệu liên quan khi thực thể cha bị xóa, tránh dữ liệu mồ côi.
- Các quan hệ cần bảo toàn dữ liệu chính như PRODUCT.Created_by_ID sử dụng ON DELETE SET NULL, cho phép giữ lại sản phẩm khi người tạo (Store Manager) bị xóa.
- Các quan hệ mang tính ràng buộc nghiệp vụ như PRODUCT.Category_ID sử dụng ON DELETE RESTRICT để ngăn việc xóa danh mục khi vẫn còn sản phẩm liên kết.
- Hệ thống sử dụng các bảng trung gian cho quan hệ nhiều - nhiều (N-N): USE (Driver - Vehicle), STORE_PRODUCT (Store - Product), ORDER_ITEM (Order - Product), CART_ITEM (Cart - Product). Các bảng này có khóa chính phức hợp hoặc khóa tự tăng kết hợp với ràng buộc UNIQUE.
- Một số khóa ngoại có sử dụng ON UPDATE CASCADE để đảm bảo tính nhất quán khi khóa chính thay đổi, ví dụ: các khóa ngoại tham chiếu đến CATEGORY, USER, PRODUCT.

1.1.2.3 Ràng buộc dữ liệu (Data Constraints)

CHECK: Kiểm tra giá trị hợp lệ cho các trường:

- Giới tính: Gender IN ('Male', 'Female', 'Other')
- Trạng thái người dùng: User_status IN ('Active', 'Inactived', 'Banned', 'Unverify')
- Trạng thái tài xế: Driver_status IN ('Available', 'Busy', 'Offline')
- Trạng thái xe: Status IN ('Available', 'In use', 'Maintenance')
- Trạng thái sản phẩm: status IN ('Pending', 'Approved', 'Rejected')

- Trạng thái giỏ hàng: `Cart_status IN ('Active', 'CheckedOut', 'Abandoned', 'Saved')`
- Trạng thái đơn hàng: `Order_status IN ('Pending', 'Completed', 'Cancelled')`
- Phương thức thanh toán: `Method IN ('Cash', 'Card', 'E-Wallet')`
- Trạng thái thanh toán: `Payment_status IN ('Pending', 'Paid', 'Failed', 'Refunded', 'Cancelled')`
- Trạng thái vận chuyển: `Status IN ('Pending', 'Shipping', 'Delivered')`
- Giá trị số: `Price > 0`, `Stock_quantity >= 0`, `CartItem_quantity > 0`, `Quantity > 0`, `Amount >= 0`, `Rating BETWEEN 1 AND 5`, `Admin_level >= 1`

UNIQUE: Đảm bảo tính duy nhất cho các trường:

- `Email` trong bảng `USER`
- `Tax_code` trong bảng `STORE_MANAGER`
- Cặp `(Store_ID, Product_ID)` trong bảng `STORE_PRODUCT`
- `Order_ID` trong bảng `PAYMENT`
- Cặp `(Cart_ID, Store_ID, SProduct_ID, Product_ID)` trong bảng `CART_ITEM`
- Cặp `(Order_ID, Item_ID)` trong bảng `REVIEW`

DEFAULT: Giá trị mặc định cho các trường:

- `User_status` mặc định là `'Active'`
- `status` trong bảng `PRODUCT` mặc định là `'Pending'`
- `Cart_status` mặc định là `'Active'`
- `Stock_quantity` mặc định là `0`
- `Total_deliveries` trong bảng `DRIVER` mặc định là `0`

NOT NULL: Các trường bắt buộc phải có giá trị (tất cả khóa chính, khóa ngoại và các trường quan trọng như `User_name`, `Password`, `First_name`, `Price`, `Stock_quantity`, `Quantity`, `Amount`, v.v.)

1.1.2.4 Ràng buộc ngữ nghĩa cần xử lý bằng trigger

Trong phần tạo bảng, nhóm chỉ đặt những ràng buộc có thể kiểm tra trực tiếp bằng `CHECK`, `FOREIGN KEY`, `UNIQUE` hoặc `DEFAULT`. Các ràng buộc cần đọc dữ liệu từ nhiều bảng hoặc cần phản ứng theo thao tác `INSERT`, `UPDATE`, `DELETE` được tách sang phần trigger ở mục 2.2 để đúng yêu cầu đề bài.

Các trigger được hiện thực trong file `database/trigger.sql`, gồm:

- `trg_Check_Review`: kiểm tra khách hàng chỉ được đánh giá sản phẩm khi đơn hàng tương ứng đã hoàn tất. Trigger này cần đọc đồng thời bảng `ORDER` và `ORDER_ITEM`, vì vậy không thể thay bằng `CHECK`.
- `trg_UpdateDelivery_Insert`, `trg_UpdateDelivery_Update`, `trg_UpdateDelivery_Delete`: cập nhật thuộc tính dẫn xuất `DRIVER.Total_deliveries` dựa trên các vận đơn có trạng thái `Delivered`.

Như vậy, Part 4 tập trung vào lược đồ và dữ liệu mẫu; phần code trigger chi tiết được trình bày ở Part 5 để tránh trùng lặp và để người đọc theo đúng bố cục của yêu cầu BTL2.

1.1.2.5 Các điểm đặc biệt trong thiết kế

- **Quan hệ chuyên biệt hóa:** Bảng `USER` đóng vai trò bảng cha, các bảng `ADMIN`, `STORE_MANAGER`, `CUSTOMER`, `DRIVER` là các bảng con, liên kết thông qua khóa ngoại tham chiếu đến `User_ID`. Cách thiết kế này cho phép phân tách dữ liệu theo từng loại người dùng trong khi vẫn đảm bảo tính thống nhất.
- **Quan hệ phân cấp danh mục:** Bảng `CATEGORY` quản lý danh mục sản phẩm. Cấu trúc phân cấp được thể hiện thông qua bảng `SUB_CATEGORY`, trong đó mỗi bản ghi liên kết một danh mục cha với một danh mục con, cho phép xây dựng cây danh mục nhiều cấp độ.
- **Quan hệ nhiều - nhiều (N-N):** Hệ thống sử dụng các bảng trung gian để thể hiện quan hệ nhiều - nhiều giữa các thực thể, ví dụ: `USE` (Driver - Vehicle). Ngoài ra, các bảng như `STORE_PRODUCT`, `ORDER_ITEM`, `CART_ITEM` đóng vai trò liên kết giữa các thực thể và có thể lưu thêm các thuộc tính như giá và số lượng.
- **Ràng buộc xóa dây chuyền:** Ràng buộc `ON DELETE CASCADE` được sử dụng cho các bảng phụ thuộc như `USER_PHONE`, `CART`, `ADDRESS`, `STORE_PRODUCT` nhằm đảm bảo dữ liệu liên quan được xóa đồng bộ, tránh dữ liệu mồ côi.

1.1.2.6 Kết quả thực thi Sau khi chạy script tạo bảng và dữ liệu mẫu, hệ thống báo cáo:

- Tổng số bảng: 25 bảng
- Các bảng chính như `USER`, `PRODUCT`, `STORE`, `ORDER`, `ORDER_ITEM`, `PAYMENT`, `SHIPMENT`, `REVIEW` được tạo thành công.
- Các ràng buộc khóa chính, khóa ngoại và `CHECK` trong script tạo bảng được MySQL chấp nhận khi chạy thử với dữ liệu mẫu.

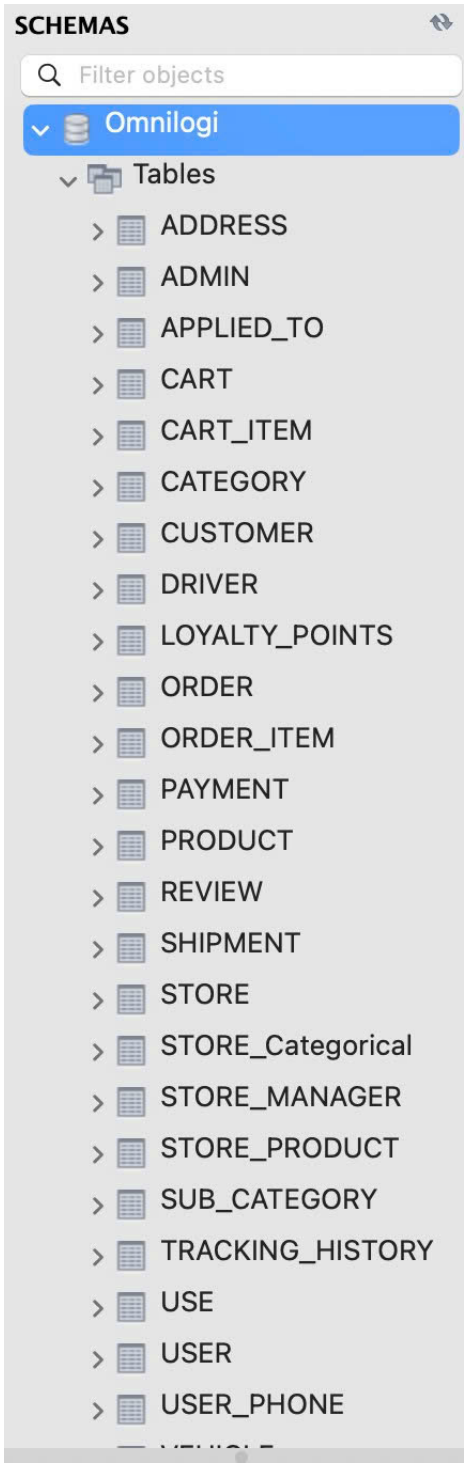
1.1.2.7 Code SQL – Đoạn trích tạo bảng chính Dưới đây là đoạn code SQL tạo các bảng `USER`, `PRODUCT` và `ORDER_ITEM`. Đây là ba bảng tiêu biểu cho nhóm người dùng, sản phẩm và giao dịch đặt hàng.

```sql
-- Bang USER
CREATE TABLE USER (
    User_ID INT AUTO_INCREMENT PRIMARY KEY,
    User_Role VARCHAR(50) NOT NULL,
    User_name VARCHAR(50) NOT NULL,
    `Password` VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    First_name NVARCHAR(50) NOT NULL,
    Middle_name NVARCHAR(50),
    Last_name NVARCHAR(50),
    Gender VARCHAR(10),
    Date_of_birth DATE,
    User_status NVARCHAR(20),
    Created_date DATETIME,
    Admin_ID INT NULL,
    CHECK (Gender IN ('Male', 'Female', 'Other')),
    CHECK(User_status IN('Active', 'Inactived', 'Banned', 'Unverify'))
);

-- Bang PRODUCT
CREATE TABLE PRODUCT (
    Product_ID INT AUTO_INCREMENT PRIMARY KEY,
    Category_ID INT NOT NULL,
    `Name` VARCHAR(100) NOT NULL,
```

```
    `Description` TEXT,
    Brand VARCHAR(50),
    image_url VARCHAR(255),
    status VARCHAR(50) DEFAULT 'Pending',
    Created_by_ID INT,
    CHECK (status IN ('Pending', 'Approved', 'Rejected')),
    FOREIGN KEY (Category_ID) REFERENCES CATEGORY(Category_ID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (Created_by_ID) REFERENCES STORE_MANAGER(User_ID)
        ON DELETE SET NULL ON UPDATE CASCADE
);

-- Bang ORDER_ITEM
CREATE TABLE ORDER_ITEM (
    Item_ID INT AUTO_INCREMENT NOT NULL,
    Order_ID INT NOT NULL,
    PRIMARY KEY(Item_ID, Order_ID),
    SProduct_ID INT NOT NULL,
    Store_ID INT NOT NULL,
    Product_ID INT NOT NULL,
    Quantity INT CHECK (Quantity > 0),
    Price_of_purchase DECIMAL(10,2),
    FOREIGN KEY (Order_ID) REFERENCES `ORDER`(Order_ID),
    FOREIGN KEY (SProduct_ID) REFERENCES STORE_PRODUCT(SProduct_ID),
    FOREIGN KEY (Store_ID) REFERENCES STORE(Store_ID),
    FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID)
);
```



Hình 1.1: Kết quả sau khi tạo 25 bảng

1.2 Tạo dữ liệu mẫu (Sample Data)

1.2.1 Mô tả

Phần này thực hiện việc khởi tạo và nhập dữ liệu mẫu thực tế cho cơ sở dữ liệu `Omnilogi`. Dữ liệu bao gồm các đối tượng cốt lõi của hệ thống thương mại điện tử tích hợp logistics như: Người dùng (USER), Quản trị viên (ADMIN), Quản lý cửa hàng (STORE_MANAGER), Khách hàng (CUSTOMER), Tài xế (DRIVER), Gian hàng (STORE), Danh mục (CATEGORY), Sản phẩm (PRODUCT), Giỏ hàng (CART), Đơn hàng (ORDER), Thanh toán (PAYMENT), Vận đơn (SHIPMENT), Đánh giá (REVIEW) và các giao dịch liên quan. Mục tiêu là tạo ra một tập dữ liệu đủ lớn và có ý nghĩa logic để phục vụ việc kiểm thử các truy vấn và ràng buộc toàn vẹn.

1.2.2 Input

- **Lược đồ cơ sở dữ liệu:** Đã thiết kế ở mục 1.1 với 25 bảng và đầy đủ các ràng buộc (PK, FK, CHECK, UNIQUE, DEFAULT).
- **Bộ dữ liệu giả lập:** Dữ liệu được tạo ra mang tính thực tế, bao gồm:
  - 20 người dùng (USER): 5 ADMIN, 5 STORE_MANAGER, 5 CUSTOMER, 5 DRIVER.
  - 5 cửa hàng (STORE) với các ngành hàng khác nhau.
  - 24 sản phẩm (PRODUCT) thuộc nhiều danh mục (Điện tử, Thời trang, Thực phẩm).
  - 15 đơn hàng (ORDER) với đầy đủ các trạng thái (Pending, Completed, Cancelled).
  - Các dữ liệu liên quan: USER_PHONE, ADDRESS, CART, CART_ITEM, ORDER_ITEM, PAYMENT, SHIPMENT, TRACKING_HISTORY, REVIEW, LOYALTY_POINTS.

1.2.3 Logic hoạt động

Quá trình nhập liệu tuân thủ nghiêm ngặt thứ tự ràng buộc khóa ngoại (Foreign Key) để đảm bảo toàn vẹn dữ liệu:

1. **Khởi tạo Master Data:** Tạo Database, các bảng USER, CATEGORY, VEHICLE trước.
2. **Khởi tạo thực thể con:** Tạo ADMIN, STORE_MANAGER, CUSTOMER, DRIVER từ USER, sau đó tạo STORE và ADDRESS.
3. **Nhập liệu danh mục và sản phẩm:** Insert CATEGORY → SUB_CATEGORY → PRODUCT → STORE_PRODUCT.

4. **Nhập liệu giỏ hàng:** Insert CART → CART_ITEM.
5. **Nhập liệu giao dịch:** Insert ORDER → ORDER_ITEM → PAYMENT → SHIPMENT → TRACKING_HISTORY → REVIEW.
6. **Nhập liệu điểm thưởng:** Insert LOYALTY_POINTS → APPLIED_TO.

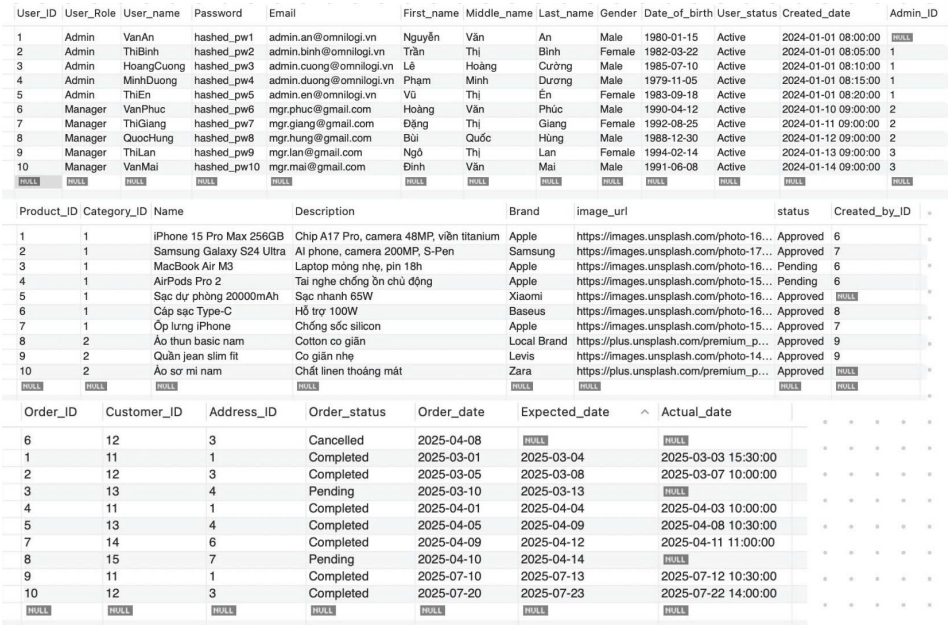
1.2.4 Output

- Cơ sở dữ liệu `Omnilogi` chứa 25 bảng có dữ liệu.
- Tổng cộng 20 `USER`, 5 `STORE`, 24 `PRODUCT`, 15 `ORDER` và các dữ liệu liên quan.
- Tất cả các ràng buộc đều được thỏa mãn, không có lỗi tham chiếu.

1.2.5 Demo

Dưới đây là kết quả truy vấn kiểm tra dữ liệu sau khi chạy script nhập liệu:

```
1
2 SELECT * FROM USER LIMIT 10;
3
4 SELECT * FROM PRODUCT LIMIT 10;
5
6 SELECT * FROM `ORDER` LIMIT 10;
```



User_ID	User_Role	User_name	Password	Email	First_name	Middle_name	Last_name	Gender	Date_of_birth	User_status	Created_date	Admin_ID
1	Admin	VanAn	hashed_pw1	admin.an@omnilogi.vn	Nguyễn	Văn	An	Male	1980-01-15	Active	2024-01-01 08:00:00	NULL
2	Admin	ThiBinh	hashed_pw2	admin.binh@omnilogi.vn	Trần	Thị	Bình	Female	1982-03-22	Active	2024-01-01 08:05:00	1
3	Admin	HoangCuong	hashed_pw3	admin.cuong@omnilogi.vn	Lê	Hoàng	Cường	Male	1985-07-10	Active	2024-01-01 08:10:00	1
4	Admin	MinhDuong	hashed_pw4	admin.duong@omnilogi.vn	Phạm	Minh	Dương	Male	1979-11-05	Active	2024-01-01 08:15:00	1
5	Admin	ThiEn	hashed_pw5	admin.en@omnilogi.vn	Vũ	Thị	Ên	Female	1983-09-18	Active	2024-01-01 08:20:00	1
6	Manager	VanPhuc	hashed_pw6	mgr.phuc@gmail.com	Hoàng	Văn	Phúc	Male	1990-04-12	Active	2024-01-10 09:00:00	2
7	Manager	ThiGiang	hashed_pw7	mgr.giang@gmail.com	Đặng	Thị	Giang	Female	1992-08-25	Active	2024-01-11 09:00:00	2
8	Manager	QuocHung	hashed_pw8	mgr.hung@gmail.com	Bùi	Quốc	Hùng	Male	1988-12-30	Active	2024-01-12 09:00:00	2
9	Manager	ThiLan	hashed_pw9	mgr.lan@gmail.com	Ngô	Thị	Lan	Female	1994-02-14	Active	2024-01-13 09:00:00	3
10	Manager	VanMai	hashed_pw10	mgr.mai@gmail.com	Đinh	Văn	Mai	Male	1991-06-08	Active	2024-01-14 09:00:00	3
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Product_ID	Category_ID	Name	Description	Brand	image_url	status	Created_by_ID
1	1	iPhone 15 Pro Max 256GB	Chip A17 Pro, camera 48MP, viền titanium	Apple	https://images.unsplash.com/photo-16...	Approved	6
2	1	Samsung Galaxy S24 Ultra	AI phone, camera 200MP, S-Pen	Samsung	https://images.unsplash.com/photo-17...	Approved	7
3	1	MacBook Air M3	Laptop mỏng nhẹ, pin 18h	Apple	https://images.unsplash.com/photo-16...	Pending	6
4	1	AirPods Pro 2	Tai nghe chống ồn chủ động	Apple	https://images.unsplash.com/photo-15...	Pending	6
5	1	Sạc dự phòng 20000mAh	Sạc nhanh 65W	Xiaomi	https://images.unsplash.com/photo-16...	Approved	NULL
6	1	Cáp sạc Type-C	Hỗ trợ 100W	Baseus	https://images.unsplash.com/photo-16...	Approved	8
7	1	Ốp lưng iPhone	Chống sốc silicon	Apple	https://images.unsplash.com/photo-15...	Approved	7
8	2	Áo thun basic nam	Cotton co giãn	Local Brand	https://plus.unsplash.com/premium_p...	Approved	9
9	2	Quần jean slim fit	Co giãn nhẹ	Levis	https://images.unsplash.com/photo-14...	Approved	9
10	2	Áo sơ mi nam	Chất linen thoáng mát	Zara	https://plus.unsplash.com/premium_p...	Approved	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Order_ID	Customer_ID	Address_ID	Order_status	Order_date	Expected_date	Actual_date
6	12	3	Cancelled	2025-04-08	NULL	NULL
1	11	1	Completed	2025-03-01	2025-03-04	2025-03-03 15:30:00
2	12	3	Completed	2025-03-05	2025-03-08	2025-03-07 10:00:00
3	13	4	Pending	2025-03-10	2025-03-13	NULL
4	11	1	Completed	2025-04-01	2025-04-04	2025-04-03 10:00:00
5	13	4	Completed	2025-04-05	2025-04-09	2025-04-08 10:30:00
7	14	6	Completed	2025-04-09	2025-04-12	2025-04-11 11:00:00
8	15	7	Pending	2025-04-10	2025-04-14	NULL
9	11	1	Completed	2025-07-10	2025-07-13	2025-07-12 10:30:00
10	12	3	Completed	2025-07-20	2025-07-23	2025-07-22 14:00:00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 1.2: Kết quả truy vấn dữ liệu mẫu các bảng USER, PRODUCT, ORDER

1.2.6 Code nhập dữ liệu mẫu

Dưới đây là trích đoạn mã SQL dùng để nhập dữ liệu mẫu cho 25 bảng trong cơ sở dữ liệu Omnilogi:

```
-- ============================================================
-- 1. USER (5 ADMIN, 5 STORE_MANAGER, 5 CUSTOMER, 5 DRIVER)
-- ============================================================
INSERT INTO USER (User_ID, User_Role, User_name, ‘Password‘, Email, First_name,
    Middle_name, Last_name,
                  Gender, Date_of_birth, User_status, Created_date, Admin_ID)
VALUES
-- Admin
(1, 'Admin', 'VanAn', 'hashed_pw1', 'admin.an@omnilogi.vn', N'Nguyen', N'Van', N'An', '
    Male', '1980-01-15', 'Active', '2024-01-01 08:00:00', NULL),
-- Store Manager
(6, 'Manager', 'VanPhuc', 'hashed_pw6', 'mgr.phuc@gmail.com', N'Hoang', N'Van', N'Phuc',
     'Male', '1990-04-12', 'Active', '2024-01-10 09:00:00', NULL),
-- Customer
(11, 'Customer', 'na_shop', 'hashed_pw11', 'na.cust@gmail.com', N'Cao', N'Thi', N'Na', '
    Female', '1998-03-20', 'Active', '2024-02-01 10:00:00', NULL),
-- Driver
(16, 'Driver', 'VanTai', 'hashed_pw16', 'tai.drv@omnilogi.vn', N'Son', N'Van', N'Tai', '
    Male', '1992-04-18', 'Active', '2024-01-05 07:00:00', NULL);

-- ============================================================
-- 2. USER_PHONE
-- ============================================================
INSERT INTO USER_PHONE (User_ID, Phone) VALUES
(1, '0901234567'),
(11, '0911223344'),
(16, '0966778899');

-- ============================================================
-- 3. ADMIN
-- ============================================================
INSERT INTO ADMIN (User_ID, Admin_level, Last_login) VALUES
```

```
(1, 5, '2025-04-10 08:00:00'),
(2, 4, '2025-04-10 08:30:00'),
(3, 3, '2025-04-09 17:00:00');

-- ============================================================
-- 4. STORE_MANAGER
-- ============================================================
INSERT INTO STORE_MANAGER (User_ID, Tax_code) VALUES
(6, '0123456789'),
(7, '0234567890'),
(8, '0345678901');

-- ============================================================
-- 5. CUSTOMER
-- ============================================================
INSERT INTO CUSTOMER (User_ID) VALUES
(11), (12), (13);

-- ============================================================
-- 6. DRIVER
-- ============================================================
INSERT INTO DRIVER (User_ID, License_num, Vehicle_type, Driver_status, Total_deliveries)
     VALUES
(16, 'BX001-2020', 'Xe may', 'Available', 3),
(17, 'BX002-2021', 'Xe may', 'Busy', 3),
(18, 'BX003-2019', 'O to', 'Available', 1);

-- ============================================================
-- 7. VEHICLE
-- ============================================================
INSERT INTO VEHICLE (Vehicle_id, Plate_number, `Type`, `Status`) VALUES
(1, '59B1-12345', 'Xe may', 'In use'),
(2, '59C2-23456', 'Xe may', 'Available'),
(3, '51A3-34567', 'O to', 'Available');

-- ============================================================
```

```
-- 8. USE (Driver - Vehicle N-M)
-- ============================================================
INSERT INTO `USE` (User_ID, Vehicle_id) VALUES
(16, 1), (16, 2),
(17, 1), (17, 5);

-- ============================================================
-- 9. CATEGORY
-- ============================================================
INSERT INTO CATEGORY (Category_ID, Category_name, Admin_ID) VALUES
(1, N'Dien tu', 1),
(2, N'Thoi trang', 1),
(3, N'Thuc pham', 2);

-- ============================================================
-- 10. SUB_CATEGORY
-- ============================================================
INSERT INTO SUB_CATEGORY (Category_ID, Sub_category_ID, Sub_category_Name) VALUES
(1, 1, N'Dien thoai'),
(1, 2, N'Laptop & May tinh'),
(2, 1, N'Ao');

-- ============================================================
-- 11. STORE
-- ============================================================
INSERT INTO STORE (Store_ID, Admin_ID, Manager_ID, `Name`, `Description`, Store_address)
    VALUES
(1, 1, 6, N'TechViet Store', N'Chuyen dien tu chinh hang', N'123 Nguyen Hue, Q1, TP.HCM'
    ),
(2, 1, 7, N'Fashion Hub', N'Thoi trang nam nu cao cap', N'45 Le Loi, Q1, TP.HCM'),
(3, 2, 8, N'Foods & More', N'Thuc pham sach', N'78 Tran Hung Dao, Q5, TP.HCM');

-- ============================================================
-- 12. STORE_Categorical
-- ============================================================
INSERT INTO STORE_Categorical (Store_ID, Categorical) VALUES
```

```
(1, N'Dien thoai & thiet bi Apple'),
(1, N'Dien thoai Samsung'),
(2, N'Thoi trang nam');

-- ============================================================
-- 13. PRODUCT
-- ============================================================
INSERT INTO PRODUCT (Product_ID, Category_ID, `Name`, `Description`, Brand, image_url,
    status, Created_by_ID) VALUES
(1, 1, 'iPhone 15 Pro Max 256GB', 'Chip A17 Pro, camera 48MP', 'Apple', 'https://...', '
    Approved', 6),
(2, 1, 'Samsung Galaxy S24 Ultra', 'AI phone, camera 200MP', 'Samsung', 'https://...', '
    Approved', 7),
(8, 2, 'Ao thun basic nam', 'Cotton co gian', 'Local Brand', 'https://...', 'Approved',
    9);

-- ============================================================
-- 14. STORE_PRODUCT
-- ============================================================
INSERT INTO STORE_PRODUCT (Store_ID, Product_ID, Price, Stock_quantity) VALUES
(1, 1, 32990000, 50),
(4, 1, 33500000, 30),
(2, 8, 99000, 200);

-- ============================================================
-- 15. ADDRESS
-- ============================================================
INSERT INTO ADDRESS (Address_ID, Customer_ID, Recipient_name, Recipient_phone,
    Address_type, Details) VALUES
(1, 11, N'Cao Thi Na', '0911223344', 'Home', N'12 Nguyen Oanh, P17, Q Go Vap, TP.HCM'),
(3, 12, N'Ly Van Oanh', '0922334455', 'Home', N'45 Dinh Tien Hoang, P3, Q Binh Thanh, TP
    .HCM'),
(4, 13, N'To Thi Phuong', '0933445566', 'Home', N'78 Vo Thi Sau, P7, Q3, TP.HCM');

-- ============================================================
-- 16. CART
```

```
-- ============================================================
INSERT INTO CART (Cart_ID, Cart_status, Customer_ID) VALUES
(1, 'Active', 11),
(2, 'Active', 12),
(3, 'Saved', 13);

-- ============================================================
-- 17. CART_ITEM
-- ============================================================
INSERT INTO CART_ITEM (Cart_ID, Store_ID, SProduct_ID, Product_ID, CartItem_quantity)
    VALUES
(1, 1, 3, 2, 1),
(2, 1, 1, 1, 1),
(3, 3, 17, 14, 2);

-- ============================================================
-- 18. ORDER
-- ============================================================
INSERT INTO `ORDER` (Customer_ID, Address_ID, Order_status, Order_date, Expected_date,
    Actual_date) VALUES
(11, 1, 'Completed', '2025-03-01', '2025-03-04', '2025-03-03 15:30:00'),
(12, 3, 'Completed', '2025-03-05', '2025-03-08', '2025-03-07 10:00:00'),
(13, 4, 'Pending', '2025-03-10', '2025-03-13', NULL);

-- ============================================================
-- 19. ORDER_ITEM
-- ============================================================
INSERT INTO ORDER_ITEM (Item_ID, Order_ID, SProduct_ID, Store_ID, Product_ID, Quantity,
    Price_of_purchase) VALUES
(1, 1, 2, 4, 1, 1, 33500000),
(2, 1, 7, 4, 7, 1, 120000),
(1, 2, 3, 1, 2, 1, 22990000);

-- ============================================================
-- 20. PAYMENT
-- ============================================================
```

```
160 INSERT INTO PAYMENT (Payment_ID, Order_ID, Method, Payment_date, Payment_status, Amount)
        VALUES
161 (1, 1, 'Card', '2025-03-01', 'Paid', 33770000),
162 (2, 2, 'E-Wallet', '2025-03-05', 'Paid', 23140000),
163 (3, 3, 'Cash', '2025-03-10', 'Pending', 6550000);
164
165 -- ============================================================
166 -- 21. SHIPMENT
167 -- ============================================================
168 INSERT INTO SHIPMENT (Shipment_ID, Driver_ID, Order_ID, Status) VALUES
169 (1, 16, 1, 'Delivered'),
170 (2, 17, 2, 'Delivered'),
171 (3, 18, 3, 'Shipping');
172
173 -- ============================================================
174 -- 22. TRACKING_HISTORY
175 -- ============================================================
176 INSERT INTO TRACKING_HISTORY (Tracking_ID, Shipment_ID, Current_location, Status_updated
        , Time_stamp) VALUES
177 (1, 1, N'Kho MobileZone - Q10, TP.HCM', N'Da lay hang', '2025-03-01 12:00:00'),
178 (2, 1, N'12 Nguyen Oanh, Go Vap, TP.HCM', N'Da giao hang', '2025-03-03 15:30:00'),
179 (1, 2, N'Kho TechViet - Q1, TP.HCM', N'Da lay hang', '2025-03-05 16:00:00');
180
181 -- ============================================================
182 -- 23. REVIEW
183 -- ============================================================
184 INSERT INTO REVIEW (Order_ID, Item_ID, Customer_ID, Rating, `Comment`, Review_date)
        VALUES
185 (1, 1, 11, 5, N'iPhone 15 Pro Max dung rat muot', '2025-03-04'),
186 (1, 2, 11, 4, N'Op lung chac chan, om may tot', '2025-03-04'),
187 (2, 1, 12, 4, N'Samsung S24 Ultra man hinh pin tot', '2025-03-08');
188
189 -- ============================================================
190 -- 24. LOYALTY_POINTS
191 -- ============================================================
192 INSERT INTO LOYALTY_POINTS (Transaction_ID, Order_ID, Customer_ID, `Transaction`,
```

```
    Expiry_date) VALUES
(1, 1, 11, 337700, '2026-03-01 00:00:00'),
(5, 2, 12, 231400, '2026-03-05 00:00:00'),
(8, 5, 13, 159900, '2026-04-05 00:00:00');

-- ============================================================
-- 25. APPLIED_TO
-- ============================================================
INSERT INTO APPLIED_TO (Order_ID, Transaction_ID) VALUES
(4, 3),
(5, 8),
(7, 11);
```

2 Viết các trigger, thủ tục, hàm

2.1 Các thủ tục thêm, xoá, sửa vào bảng Product

Theo yêu cầu mục 2.1, nhóm chọn bảng `PRODUCT` làm bảng dữ liệu trung tâm để hiện thực ba thủ tục thêm, sửa, xoá. Đây là bảng phù hợp với nghiệp vụ của hệ thống OmniLogi vì sản phẩm là đối tượng xuất hiện xuyên suốt từ danh mục, cửa hàng, giỏ hàng, đơn hàng đến đánh giá. Việc thêm, duyệt, cập nhật hoặc xoá sản phẩm nếu không kiểm soát chặt sẽ ảnh hưởng trực tiếp đến dữ liệu bán hàng và lịch sử giao dịch.

Các thủ tục thêm, sửa, xoá được viết trong file `database/crud - product.sql` và được backend gọi trực tiếp thông qua API. Cả ba thủ tục đều dùng `SIGNAL SQLSTATE '45000'` để trả về thông báo lỗi rõ nghĩa, thay vì chỉ báo lỗi chung chung.

2.1.1 Thủ tục thêm sản phẩm `sp_insert_product`

- Mục đích Thủ tục `sp_insert_product` dùng để thêm mới sản phẩm vào bảng `PRODUCT`. Nghiệp vụ được tách thành hai luồng:

- Store manager đề xuất sản phẩm mới: sản phẩm phải ở trạng thái `Pending` và bắt buộc có `Created_by_ID`.
- Admin thêm sản phẩm đã duyệt: sản phẩm có thể ở trạng thái `Approved`; nếu có store manager tạo, hệ thống tự động liên kết sản phẩm vào bảng `STORE_PRODUCT`.

- Input và Output

- **Input**: `p_category_id`, `p_name`, `p_desc`, `p_brand`, `p_image_url`, `p_status`, `p_created_by_id`.
- **Output**: `Product_ID` vừa tạo và thông báo kết quả xử lý.

- Kiểm tra ràng buộc

1. `Category_ID` không được rỗng và phải tồn tại trong bảng `CATEGORY`.
2. Tên sản phẩm không được rỗng sau khi `TRIM`.
3. Trạng thái chỉ được thuộc tập `Pending`, `Approved`, `Rejected`.
4. Nếu có `Created_by_ID`, mã này phải là một store manager hợp lệ và phải đang quản lý ít nhất một cửa hàng.

5. Nếu sản phẩm ở trạng thái `Pending`, bắt buộc phải có store manager đề xuất để phục vụ quy trình duyệt.

```
IF p_category_id IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Loi Nghiep Vu: Category_ID khong duoc de trong.';
END IF;

IF p_name IS NULL OR TRIM(p_name) = '' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Loi Nghiep Vu: Ten san pham khong duoc de trong.';
END IF;

IF p_status IS NULL OR p_status NOT IN ('Pending', 'Approved', 'Rejected') THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Loi Nghiep Vu: Trang thai chi duoc phep la Pending, Approved
    hoac Rejected.';
END IF;
```

Listing 1: Kiem tra du lieu dau vao khi them san pham

Đoạn code trên xử lý ba lỗi nhập liệu thường gặp nhất: thiếu danh mục, thiếu tên sản phẩm và trạng thái không hợp lệ. Các lỗi này được phát hiện trước khi `INSERT`, nhờ đó dữ liệu rác không đi vào bảng `PRODUCT`.

```
INSERT INTO PRODUCT (
    Category_ID, `Name`, `Description`, Brand,
    image_url, `status`, Created_by_ID
)
VALUES (
    p_category_id,
    TRIM(p_name),
    NULLIF(TRIM(p_desc), ''),
    NULLIF(TRIM(p_brand), ''),
    NULLIF(TRIM(p_image_url), ''),
    p_status,
    p_created_by_id
);
```

```
14
15 SET v_new_product_id = LAST_INSERT_ID();
16
17 IF p_status = 'Approved' AND v_store_id IS NOT NULL THEN
18     INSERT IGNORE INTO STORE_PRODUCT (Store_ID, Product_ID, Price, Stock_quantity)
19     VALUES (v_store_id, v_new_product_id, 1, 0);
20 END IF;
```

Listing 2: Them san pham va tu dong lien ket STORE_PRODUCT khi duoc duyet

Hàm `NULLIF(TRIM(...), '')` giúp chuẩn hóa chuỗi rỗng thành `NULL`. Khi sản phẩm được tạo ở trạng thái `Approved`, thủ tục tự động tạo bản ghi trong `STORE_PRODUCT` với giá mặc định `1` và tồn kho `0`; điều này giúp sản phẩm vừa được duyệt có thể xuất hiện trong luồng bán hàng nhưng vẫn yêu cầu quản lý cửa hàng cập nhật giá và tồn kho thực tế.

- Câu lệnh kiểm thử và kết quả

```
1 CALL sp_insert_product(
2     1, 'Tai nghe Sony XM5', 'Chong on tot',
3     'Sony', 'sony-xm5.jpg', 'Pending', 6
4 );
5
6 CALL sp_insert_product(
7     999, 'San pham loi', 'Sai danh muc',
8     'Sony', 'link.jpg', 'Pending', 6
9 );
```

Listing 3: Kiem thu them san pham

Kết quả chạy thủ tục:

- Trường hợp hợp lệ trả về `Product_ID` mới và thông báo `Đã tạo đề xuất sản phẩm ở trạng thái Pending.`
- Trường hợp sai danh mục bị chặn với lỗi `Lỗi Nghiệp Vụ: Danh mục (Category_ID) không tồn tại.`

2.1.2 Thủ tục sửa sản phẩm `sp_update_product`

- Mục đích Thủ tục `sp_update_product` dùng để cập nhật thông tin sản phẩm và hỗ trợ quy trình duyệt sản phẩm. Ngoài việc sửa tên, mô tả, thương hiệu, ảnh và danh mục, thủ tục còn xử lý nghiệp vụ khi sản phẩm chuyển sang hoặc rời khỏi trạng thái `Approved`.

- Logic xử lý chính

1. Kiểm tra `Product_ID` tồn tại.
2. Nếu truyền `Category_ID` mới thì danh mục mới phải tồn tại.
3. Nếu truyền tên mới thì tên không được là chuỗi rỗng.
4. Nếu truyền trạng thái mới thì trạng thái phải hợp lệ.
5. Nếu chuyển từ chưa duyệt sang `Approved`, tự động tạo liên kết `STORE_PRODUCT`.
6. Nếu chuyển từ `Approved` sang trạng thái khác, chỉ cho phép khi sản phẩm chưa phát sinh đơn hàng hoặc giỏ hàng.

- Đoạn code quan trọng

```
IF v_old_status = 'Approved' AND v_new_status <> 'Approved' THEN
    SELECT COUNT(*)
    INTO v_order_item_count
    FROM ORDER_ITEM
    WHERE Product_ID = p_product_id;

    IF v_order_item_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Loi Nghiep Vu: San pham da phat sinh don hang nen khong the
    chuyen khoi trang thai Approved.';
    END IF;

    SELECT COUNT(*)
    INTO v_cart_item_count
    FROM CART_ITEM
    WHERE Product_ID = p_product_id;

    IF v_cart_item_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Loi Nghiep Vu: San pham dang ton tai trong gio hang nen
    khong the chuyen khoi trang thai Approved.';
    END IF;
END IF;
```

Listing 4: Chan huy duyet san pham da phat sinh giao dich

Đây là phần quan trọng nhất của thủ tục sửa. Một sản phẩm đã được duyệt có thể đã xuất hiện trong đơn hàng, giỏ hàng hoặc lịch sử mua bán. Nếu cho phép đổi trạng thái tự do, hệ thống có thể mất tính nhất quán giữa màn hình sản phẩm và dữ liệu giao dịch. Vì vậy thủ tục chỉ cho phép hủy duyệt khi sản phẩm chưa có dữ liệu phụ thuộc.

```
UPDATE PRODUCT
SET Category_ID = COALESCE(p_category_id, Category_ID),
    `Name` = CASE
        WHEN p_name IS NULL THEN `Name`
        ELSE TRIM(p_name)
    END,
    `Description` = CASE
        WHEN p_desc IS NULL THEN `Description`
        ELSE NULLIF(TRIM(p_desc), '')
    END,
    Brand = CASE
        WHEN p_brand IS NULL THEN Brand
        ELSE NULLIF(TRIM(p_brand), '')
    END,
    image_url = CASE
        WHEN p_image_url IS NULL THEN image_url
        ELSE NULLIF(TRIM(p_image_url), '')
    END,
    `status` = v_new_status
WHERE Product_ID = p_product_id;
```

Listing 5: Cap nhat tung truong bang COALESCE va CASE

`COALESCE` và `CASE` cho phép cập nhật một phần: trường nào không được truyền vào sẽ giữ nguyên giá trị cũ. Cách này phù hợp với API `PUT /api/products/:id`, vì frontend có thể gửi các trường cần sửa mà không phá vỡ dữ liệu còn lại.

- Câu lệnh kiểm thử và kết quả

```
CALL sp_update_product(25, NULL, NULL, NULL, NULL, NULL, 'Approved');

```

```
CALL sp_update_product(
    1, 1, 'iPhone 15 Pro Max 256GB VN/A',
    NULL, 'Apple', NULL, NULL
);
```

Listing 6: Kiem thu sua san pham

Kết quả chạy thủ tục:

- Khi duyệt sản phẩm hợp lệ, thủ tục trả về `New_Status = Approved` và thông báo đã tạo liên kết `STORE_PRODUCT` nếu có store manager tương ứng.
- Khi cập nhật thông tin sản phẩm, thủ tục trả về `Đã cập nhật sản phẩm thành công.`
- Nếu cố hủy duyệt sản phẩm đã có đơn hàng, thủ tục trả lỗi cụ thể về việc sản phẩm đã phát sinh đơn hàng.

2.1.3 Thủ tục xoá sản phẩm `sp_delete_product`

- Mục đích Thủ tục `sp_delete_product` dùng để xoá các sản phẩm nháp hoặc sản phẩm bị từ chối. Nhóm không cho phép xoá sản phẩm đã `Approved`, vì sản phẩm đã duyệt có thể là một phần của lịch sử kinh doanh. Trong thực tế, sản phẩm đã bán nên được ẩn, ngừng bán hoặc hủy duyệt có kiểm soát thay vì xoá vật lý.

- Điều kiện được xoá

- Sản phẩm tồn tại trong bảng `PRODUCT`.
- Trạng thái sản phẩm là `Pending` hoặc `Rejected`.
- Sản phẩm chưa được liên kết với cửa hàng trong `STORE_PRODUCT`.
- Sản phẩm chưa xuất hiện trong `CART_ITEM`.
- Sản phẩm chưa xuất hiện trong `ORDER_ITEM`.

- Đoạn code quan trọng

```
IF v_status NOT IN ('Pending', 'Rejected') THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Loi Nghiep Vu: Chi duoc xoa san pham dang Pending hoac Rejected.
     San pham da Approved nen chi nen an/huy duyet de bao toan lich su.';
```

```
END IF;

SELECT COUNT(*)
INTO v_cnt
FROM ORDER_ITEM
WHERE Product_ID = p_product_id;

IF v_cnt > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Loi Nghiep Vu: San pham da phat sinh lich su dat hang, khong
    duoc xoa.';
END IF;
```

Listing 7: Kiem tra dieu kien truoc khi xoa san pham

Đoạn code trên thể hiện mục đích của thủ tục xoá: chỉ dọn dữ liệu chưa đi vào nghiệp vụ chính. Khi sản phẩm đã có lịch sử đặt hàng, xoá sản phẩm sẽ làm mất khả năng truy vết đơn hàng và báo cáo doanh thu.

- Câu lệnh kiểm thử và kết quả

```
CALL sp_insert_product(1, 'Ao nhap', 'San pham test', 'Brand', 'img.png', 'Pending', 6);
CALL sp_delete_product(LAST_INSERT_ID());

CALL sp_delete_product(1);
```

Listing 8: Kiem thu xoa san pham

Kết quả chạy thủ tục:

- Sản phẩm nháp vừa tạo được xoá thành công và trả về `Đã xóa sản phẩm thành công.`
- Sản phẩm đã duyệt hoặc đã phát sinh nghiệp vụ bị chặn với thông báo lỗi cụ thể.

2.2 Triggers

2.2.1 Trigger tác nhân

- Mục đích

Trigger `trg_Check_Review` được tạo ra nhằm đảm bảo ràng buộc nghiệp vụ: **chỉ những khách hàng đã nhận hàng thành công mới được phép đánh giá sản phẩm**. Dữ liệu

đánh giá luôn gắn với đơn hàng và sản phẩm thực tế mà khách hàng đã mua. Ràng buộc này không thể thực hiện bằng CHECK constraint thông thường vì cần truy vấn dữ liệu từ nhiều bảng khác nhau.

- Cơ chế hoạt động:

- **Thời điểm kích hoạt**: BEFORE INSERT trên bảng REVIEW.
- **Phạm vi**: Mỗi dòng dữ liệu được chèn vào bảng REVIEW.
- **Logic kiểm tra**:
  1. Trigger thực hiện truy vấn kiểm tra sự tồn tại của bản ghi trong bảng ORDER và ORDER_ITEM thỏa mãn:
     - Item_ID và Order_ID khớp với dữ liệu đang insert.
     - Customer_ID khớp.
     - Order_status = 'Completed'.
  2. Nếu truy vấn trả về kết quả (tồn tại đơn hàng hoàn tất): cho phép insert.
  3. Nếu không tồn tại: trigger phát lỗi bằng lệnh SIGNAL SQLSTATE '45000' với thông báo 'Loi: Ban chi co the danh gia san pham khi don hang da hoan tat!' và ngăn chặn việc insert.

- Code trigger:

```
DROP TRIGGER IF EXISTS trg_Check_Review$$
CREATE TRIGGER trg_Check_Review
BEFORE INSERT ON REVIEW
FOR EACH ROW
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM `ORDER` o
        JOIN ORDER_ITEM oi ON o.Order_ID = oi.Order_ID
        WHERE oi.Item_ID = NEW.Item_ID
          AND oi.Order_ID = NEW.Order_ID
          AND o.Customer_ID = NEW.Customer_ID
          AND o.Order_status = 'Completed'
    ) THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Loi: Ban chi co the danh gia san pham khi don hang da hoan
    tat!';
    END IF;
END$$
```

- Kiểm thử trigger

Để kiểm tra hoạt động của trigger, nhóm thực hiện hai test case trong thủ tục `sp_test_trigger`:

+ Test: Trường hợp hợp lệ (PASS)

- Tìm một cặp `(Order_ID, Item_ID)` chưa có đánh giá và đơn hàng có trạng thái `Completed`.
- Tiến hành insert review cho cặp đó.
- **Kết quả**: Insert thành công, dữ liệu được thêm vào bảng `REVIEW`.

+ Test: Trường hợp không hợp lệ (FAIL)

- Cố tình insert review với `Item_ID = 9999` không tồn tại trong đơn hàng `Order_ID = 2`.
- **Kết quả**: Trigger phát hiện lỗi, phát sinh ngoại lệ và ngăn chặn việc insert.

- Kết quả kiểm thử

Review_ID	Order_ID	Item_ID	Customer_ID	Rating	Comment	Image	Review_date
27	10	2	12	5	Test review hop le	NULL	2026-05-02

Hình 2.1: Kết quả test case hợp lệ (PASS)

```
313         -- Insert sai (Item không thuộc order)
314         INSERT INTO REVIEW (Review_ID, Item_ID, Order_ID, Customer_ID, Rating)
315         VALUES (101, 9999, 2, 16, 5);
316     END;
317
```

100% 32:315

Action Output

	Time	Action	Response	Durati
182	09:07:31	-- GỌI CHẠY TEST CALL sp_test_trig...	1 row(s) returned	- / 0.0(
183	09:07:31	DROP PROCEDURE IF EXISTS sp_tes...	0 row(s) affected	0.002(
184	09:08:28	INSERT INTO REVIEW (Review_ID, Ite...	Error Code: 1644. Loi: Ban chi co the danh gia san pham khi don hang da hoan tat!	0.0011

Hình 2.2: Kết quả test case không hợp lệ (FAIL)

2.2.2 Trigger tính thuộc tính dẫn xuất

DML Trigger được gọi khi người dùng thực hiện các thao tác hiệu chỉnh dữ liệu như INSERT, UPDATE, DELETE. Trong nghiệp vụ quản lý vận chuyển của sàn thương mại điện tử, cần tự động cập nhật số lượng đơn hàng đã giao thành công của mỗi tài xế mỗi khi có sự thay đổi về trạng thái đơn hàng trong bảng `SHIPMENT`.

Nhóm xác định sử dụng ba trigger trên bảng `SHIPMENT` để tự động cập nhật cột `Total_deliveries` trong bảng `DRIVER`:

- **Trigger INSERT**: Sau khi thêm một shipment mới, nếu shipment có trạng thái `Delivered`, trigger sẽ đếm lại toàn bộ số đơn `Delivered` của tài xế đó và cập nhật vào bảng `DRIVER`.
- **Trigger UPDATE**: Khi trạng thái của shipment thay đổi, trigger sẽ xử lý:
  - Nếu trạng thái chuyển từ khác thành `Delivered`: tăng `Total_deliveries` lên 1.
  - Nếu trạng thái chuyển từ `Delivered` sang khác: giảm `Total_deliveries` xuống 1.
- **Trigger DELETE**: Khi xóa một shipment có trạng thái `Delivered`, trigger tự động giảm `Total_deliveries` của tài xế đó xuống 1.

Trong phạm vi dữ liệu demo, nhóm chỉ cập nhật trạng thái vận đơn, không đổi tài xế phụ trách sau khi vận đơn đã được tạo. Nếu mở rộng nghiệp vụ cho phép chuyển một vận đơn `Delivered` từ tài xế này sang tài xế khác, trigger `UPDATE` cần bổ sung nhánh xử lý khi `OLD.Driver_ID <> NEW.Driver_ID` để trừ số đơn của tài xế cũ và cộng cho tài xế mới. Việc nêu rõ giả định này giúp phần trigger khớp với cách ứng dụng hiện tại đang sử dụng dữ liệu.

Demo:

Shipment_ID	Driver_ID	Order_ID	Status
1	16	1	Delivered
7	16	8	Pending
9	16	10	Delivered
14	16	15	Delivered

Hình 2.3: Thông tin các shipment của tài xế trước khi insert

Shipment_ID	Driver_ID	Order_ID	Status
1	16	1	Delivered
7	16	8	Pending
9	16	10	Delivered
14	16	15	Delivered
15	16	11	Delivered

Hình 2.4: Thông tin các shipment của tài xế sau khi insert

BEFORE_Total_deliveries	AFTER_Total_deliveries
3	4

Hình 2.5: Kết quả kiểm tra Trigger INSERT: Giá trị Total_deliveries trước và sau khi thêm shipment

Shipment_ID	Driver_ID	Order_ID	Status
3	18	3	Shipping
11	18	12	Delivered
16	18	13	Shipping

Hình 2.6: Thông tin các shipment của tài xế trước khi cập nhật trạng thái thành Delivered

Shipment_ID	Driver_ID	Order_ID	Status
3	18	3	Shipping
11	18	12	Delivered
16	18	13	Delivered

Hình 2.7: Thông tin các shipment của tài xế sau khi cập nhật trạng thái thành Delivered

Shipment_ID_Used	BEFORE_Total_deliveries	AFTER_Total_deliveries
16	1	2

Hình 2.8: Kết quả kiểm tra Trigger UPDATE: Giá trị Total_deliveries trước và sau khi cập nhật shipment thành Delivered

Shipment_ID	Driver_ID	Order_ID	Status
2	17	2	Delivered
6	17	7	Delivered
10	17	11	Delivered
16	17	2	Delivered

Hình 2.9: Thông tin các shipment của tài xế trước khi cập nhật từ Delivered sang trạng thái khác

Shipment_ID	Driver_ID	Order_ID	Status
2	17	2	Delivered
6	17	7	Delivered
10	17	11	Delivered
16	17	2	Pending

Hình 2.10: Thông tin các shipment của tài xế sau khi cập nhật từ Delivered sang trạng thái khác

Shipment_ID_Used	BEFORE_Total_Deliveries	AFTER_Total_Deliveries
16	4	3

Hình 2.11: Kết quả kiểm tra Trigger UPDATE: Giá trị Total_deliveries trước và sau khi chuyển shipment từ Delivered sang trạng thái khác

Shipment_ID	Driver_ID	Order_ID	Status
5	20	5	Delivered
8	20	9	Delivered
13	20	14	Delivered
16	20	5	Delivered

Hình 2.12: Thông tin các shipment của tài xế trước khi xoá shipment Delivered

Shipment_ID	Driver_ID	Order_ID	Status
5	20	5	Delivered
8	20	9	Delivered
13	20	14	Delivered

Hình 2.13: Thông tin các shipment của tài xế trước khi xoá shipment Delivered

Shipment_ID_Delet...	BEFORE_Total_Diliveries	AFTER_Total_Deliveries
16	4	3

Hình 2.14: Kết quả kiểm tra Trigger DELETE: Giá trị Total_deliveries trước và sau khi xóa shipment Delivered

```
-- INSERT
DROP TRIGGER IF EXISTS trg_UpdateDelivery_Insert$$
CREATE TRIGGER trg_UpdateDelivery_Insert
AFTER INSERT ON SHIPMENT
FOR EACH ROW
BEGIN
    IF NEW.Driver_ID IS NOT NULL AND NEW.‘Status‘ = ’Delivered’ THEN
        UPDATE DRIVER
        SET Total_deliveries = (
            SELECT COUNT(*)
            FROM SHIPMENT s
            WHERE s.Driver_ID = DRIVER.User_ID
              AND s.‘Status‘ = ’Delivered’
        )
        WHERE User_ID = NEW.Driver_ID;
    END IF;
END$$

-- UPDATE
DROP TRIGGER IF EXISTS trg_UpdateDelivery_Update$$
```

```sql
CREATE TRIGGER trg_UpdateDelivery_Update
AFTER UPDATE ON SHIPMENT
FOR EACH ROW
BEGIN
    -- Neu doi tu trang thai khac sang Delivered -> Tang 1
    IF (OLD.Status <> 'Delivered' OR OLD.Status IS NULL) AND NEW.Status = 'Delivered'
    THEN
        UPDATE DRIVER SET Total_deliveries = Total_deliveries + 1
        WHERE User_ID = NEW.Driver_ID;

    -- Neu dang Delivered ma bi doi nguoc lai (do loi nhap lieu) -> Giam 1
    ELSEIF OLD.Status = 'Delivered' AND NEW.Status <> 'Delivered' THEN
        UPDATE DRIVER SET Total_deliveries = Total_deliveries - 1
        WHERE User_ID = NEW.Driver_ID;
    END IF;
END$$

-- DELETE
DROP TRIGGER IF EXISTS trg_UpdateDelivery_Delete$$
CREATE TRIGGER trg_UpdateDelivery_Delete
AFTER DELETE ON SHIPMENT
FOR EACH ROW
BEGIN
    IF OLD.Status = 'Delivered' THEN
        UPDATE DRIVER SET Total_deliveries = Total_deliveries - 1
        WHERE User_ID = OLD.Driver_ID;
    END IF;
END$$
```

2.3 Viết các thủ tục

Theo yêu cầu mục 2.3, nhóm xây dựng hai thủ tục chỉ chứa các câu truy vấn hiển thị dữ liệu, có tham số đầu vào nằm trong mệnh đề `WHERE` và `HAVING`. Hai thủ tục này đều liên quan đến bảng `PRODUCT` ở mục 2.1 hoặc dữ liệu bán sản phẩm.

2.3.1 Thủ tục hiển thị danh sách sản phẩm sp_get_all_products

- Mục đích Thủ tục sp_get_all_products dùng để hiển thị danh sách sản phẩm trên giao diện trang chủ và giao diện quản lý sản phẩm. Thủ tục hỗ trợ lọc theo trạng thái, lọc theo danh mục và tìm kiếm theo từ khóa. Đây là thủ tục được frontend gọi ở màn hình danh sách sản phẩm, đồng thời liên quan trực tiếp đến bảng PRODUCT của mục 2.1.

- Input và Output

- **Input**: p_status, p_category_id, p_search_string.
- **Output**: danh sách sản phẩm gồm mã sản phẩm, tên, thương hiệu, danh mục, trạng thái, ảnh, mô tả, người đề xuất và số cửa hàng đang bán.

- Phân tích truy vấn

- Truy vấn liên kết nhiều hơn hai bảng: PRODUCT, CATEGORY, STORE_MANAGER, USER, STORE_PRODUCT.
- Mệnh đề WHERE nhận tham số lọc từ người dùng.
- Hàm tổng hợp COUNT(DISTINCT sp.Store_ID) cho biết số cửa hàng đang bán sản phẩm.
- Mệnh đề GROUP BY gom theo từng sản phẩm để tính số cửa hàng.
- Mệnh đề ORDER BY p.Product_ID DESC đưa sản phẩm mới nhất lên đầu danh sách.

- Đoạn code quan trọng

```
SELECT
    p.Product_ID,
    p.‘Name‘ AS Ten_San_Pham,
    p.Brand AS Thuong_Hieu,
    c.Category_name AS Danh_Muc,
    p.‘status‘ AS Trang_Thai,
    p.image_url,
    p.‘Description‘ AS Mo_Ta,
    COALESCE(CONCAT_WS(’ ’, u.First_name, u.Middle_name, u.Last_name), ’Admin he thong’)
     AS Nguoi_De_Xuat,
    COUNT(DISTINCT sp.Store_ID) AS So_Cua_Hang_Ban
FROM PRODUCT p
JOIN CATEGORY c ON p.Category_ID = c.Category_ID
```

```
LEFT JOIN STORE_MANAGER sm ON p.Created_by_ID = sm.User_ID
LEFT JOIN USER u ON sm.User_ID = u.User_ID
LEFT JOIN STORE_PRODUCT sp ON p.Product_ID = sp.Product_ID
WHERE
    (p_status IS NULL OR p.`status` = p_status)
    AND (p_category_id IS NULL OR p.Category_ID = p_category_id)
    AND (
        v_search_keyword IS NULL
        OR p.`Name` COLLATE utf8mb4_0900_as_ci LIKE CONCAT('%', v_search_keyword, '%')
        OR COALESCE(p.Brand, '') COLLATE utf8mb4_0900_as_ci LIKE CONCAT('%',
    v_search_keyword, '%')
        OR COALESCE(p.`Description`, '') COLLATE utf8mb4_0900_as_ci LIKE CONCAT('%',
    v_search_keyword, '%')
        OR c.Category_name COLLATE utf8mb4_0900_as_ci LIKE CONCAT('%', v_search_keyword,
     '%')
    )
GROUP BY p.Product_ID, p.`Name`, p.Brand, c.Category_name,
         p.`status`, p.image_url, p.`Description`,
         u.First_name, u.Middle_name, u.Last_name
ORDER BY p.Product_ID DESC;
```

Listing 9: Truy van danh sach san pham co filter va search

Điểm quan trọng của thủ tục là cách viết điều kiện `(tham_số IS NULL OR cột = tham_số)`. Nhờ đó cùng một thủ tục có thể phục vụ nhiều tình huống: lấy tất cả sản phẩm, lọc theo trạng thái, lọc theo danh mục, hoặc tìm kiếm theo tên/thương hiệu/mô tả/danh mục.

- Kiểm thử và kết quả

```
CALL sp_get_all_products('Approved', NULL, NULL);
CALL sp_get_all_products('Approved', NULL, 'Ao');
CALL sp_get_all_products(NULL, NULL, NULL);
```

Listing 10: Cau lenh goi thu tuc danh sach san pham

Kết quả chạy thủ tục:

- Câu lệnh thứ nhất trả về các sản phẩm đã được duyệt để hiển thị cho người mua.
- Câu lệnh thứ hai trả về các sản phẩm có từ khóa **Áo** trong tên, thương hiệu, mô tả hoặc

danh mục.

- Câu lệnh thứ ba trả về toàn bộ sản phẩm để phục vụ màn hình quản lý.

2.3.2 Thủ tục thống kê doanh thu cửa hàng `sp_get_store_revenue_stats`

- Mục đích Thủ tục `sp_get_store_revenue_stats` dùng để thống kê doanh thu của các cửa hàng theo dữ liệu đơn hàng đã hoàn thành. Thủ tục này minh họa câu truy vấn có liên kết nhiều bảng, hàm tổng hợp, `GROUP BY`, `HAVING`, `WHERE` và `ORDER BY`.

- Input và Output

- **Input**: `p_store_id`, `p_from_date`, `p_to_date`, `p_min_revenue`.
- **Output**: mã cửa hàng, tên cửa hàng, quản lý, tổng đơn hoàn thành, tổng số lượng đã bán, tổng doanh thu và điểm đánh giá trung bình.

- Phân tích truy vấn

- Truy vấn liên kết các bảng `STORE`, `STORE_MANAGER`, `USER`, `STORE_PRODUCT`, `ORDER_ITEM`, `ORDER`, `REVIEW`.
- Mệnh đề `WHERE` lọc theo cửa hàng và khoảng ngày đặt hàng.
- `COUNT(DISTINCT o.Order_ID)` đếm số đơn hoàn thành.
- `SUM(oi.Quantity)` tính tổng sản phẩm đã bán.
- `SUM(oi.Quantity * oi.Price_of_purchase)` tính doanh thu thực tế theo giá lúc mua.
- `AVG(r.Rating)` tính điểm đánh giá trung bình.
- `HAVING` lọc theo ngưỡng doanh thu sau khi đã tổng hợp.

- Đoạn code quan trọng

```
SELECT
    s.Store_ID AS Ma_Cua_Hang,
    s.‘Name‘ AS Ten_Cua_Hang,
    CONCAT_WS(' ', u.First_name, u.Middle_name, u.Last_name) AS Quan_Ly,
    COUNT(DISTINCT o.Order_ID) AS Tong_Don_Hoan_Thanh,
    SUM(oi.Quantity) AS Tong_SL_Da_Ban,
    SUM(oi.Quantity * oi.Price_of_purchase) AS Tong_Doanh_Thu_VND,
```

```
    COALESCE(ROUND(AVG(r.Rating), 1), 0) AS Diem_DG_TB
FROM STORE s
JOIN STORE_MANAGER sm ON sm.User_ID = s.Manager_ID
JOIN USER u ON u.User_ID = sm.User_ID
JOIN STORE_PRODUCT sp ON sp.Store_ID = s.Store_ID
JOIN ORDER_ITEM oi
    ON oi.SProduct_ID = sp.SProduct_ID
   AND oi.Store_ID = s.Store_ID
JOIN `ORDER` o
    ON o.Order_ID = oi.Order_ID
   AND o.Order_status = 'Completed'
LEFT JOIN REVIEW r
    ON r.Order_ID = oi.Order_ID
   AND r.Item_ID = oi.Item_ID
WHERE
    (p_store_id IS NULL OR s.Store_ID = p_store_id)
    AND (p_from_date IS NULL OR o.Order_date >= p_from_date)
    AND (p_to_date IS NULL OR o.Order_date <= p_to_date)
GROUP BY s.Store_ID, s.`Name`, u.First_name, u.Middle_name, u.Last_name
HAVING
    (p_min_revenue IS NULL OR SUM(oi.Quantity * oi.Price_of_purchase) >= p_min_revenue)
ORDER BY
    SUM(oi.Quantity * oi.Price_of_purchase) DESC,
    s.Store_ID ASC;
```

Listing 11: Truy van thong ke doanh thu cua hang

Thủ tục dùng `Price_of_purchase` trong `ORDER_ITEM` thay vì giá hiện tại trong `STORE_PRODUCT`. Đây là lựa chọn đúng về mặt nghiệp vụ vì doanh thu phải được tính theo giá tại thời điểm khách hàng mua hàng, không bị thay đổi khi cửa hàng cập nhật giá sản phẩm sau này.

- Kiểm thử và kết quả

```
CALL sp_get_store_revenue_stats(NULL, NULL, NULL, NULL);
CALL sp_get_store_revenue_stats(NULL, '2025-03-01', '2025-03-31', 20000000);
```

Listing 12: Cau lenh goi thu tuc thong ke doanh thu

Kết quả chạy thủ tục:

- Câu lệnh thứ nhất trả về bảng xếp hạng doanh thu của tất cả cửa hàng, sắp xếp giảm dần theo doanh thu.
- Câu lệnh thứ hai chỉ trả về các cửa hàng có doanh thu từ `20.000.000` VNĐ trở lên trong tháng 03/2025.
- Nếu truyền `Store_ID` không tồn tại hoặc ngày bắt đầu lớn hơn ngày kết thúc, thủ tục trả thông báo lỗi cụ thể.

2.4 Các hàm (Functions)

Hệ thống triển khai hai hàm chính: hàm gợi ý sản phẩm dựa trên độ tin cậy (confidence) và hàm phân hạng khách hàng dựa trên tổng chi tiêu.

2.4.1 Hàm gợi ý sản phẩm (fn_recommend_products)

- Mục đích: Hàm `fn_recommend_products` gợi ý các sản phẩm thường được mua cùng với sản phẩm đầu vào dựa trên phân tích lịch sử đơn hàng, sử dụng chỉ số độ tin cậy (confidence).

- Cách tính độ tin cậy: Độ tin cậy được tính theo công thức:

$$\text{confidence} = \frac{\text{số đơn hàng có chứa cả sản phẩm A và sản phẩm B}}{\text{tổng số đơn hàng có chứa sản phẩm A}} \times 100\%$$

- Input và Output

- **Input**:
  - `p_product_id INT`: Mã sản phẩm cần gợi ý.
  - `p_min_confidence DECIMAL(5,2)`: Ngưỡng độ tin cậy tối thiểu (%).
- **Output**: Chuỗi ký tự (TEXT) liệt kê các sản phẩm gợi ý kèm độ tin cậy, mỗi sản phẩm trên một dòng.

- Logic xử lý

1. Kiểm tra tham số đầu vào: `product_id` không được NULL và phải tồn tại trong bảng `PRODUCT`.
2. Tính tổng số đơn hàng có chứa sản phẩm đầu vào.
3. Sử dụng `CURSOR` để duyệt qua các sản phẩm được mua cùng, tính độ tin cậy.

4. Lọc các sản phẩm có độ tin cậy ≥ ngưỡng cho phép.

5. Trả về chuỗi kết quả với định dạng: `- Tên sản phẩm (độ tin cậy%)`.

- Demo

```
SELECT fn_recommend_products(1, 40) AS Recommendation_Result;
```

Recommendation_Result
- Cáp sạc Type-C (100.0%) - Ốp lưng iPhone (66.7%)

Hình 2.15: Kết quả gợi ý sản phẩm cho Product_ID = 1 với độ tin cậy ≥ 40%

- Mã nguồn

```
DROP FUNCTION IF EXISTS fn_recommend_products;
DELIMITER //

CREATE FUNCTION fn_recommend_products(p_product_id INT, p_min_confidence DECIMAL(5,2))
RETURNS TEXT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_related_product_id INT;
    DECLARE v_related_product_name VARCHAR(100);
    DECLARE v_occurrence_count INT;
    DECLARE v_total_orders INT DEFAULT 0;
    DECLARE v_confidence DECIMAL(5,2);
    DECLARE v_result TEXT DEFAULT '';
    DECLARE v_count INT DEFAULT 0;
    DECLARE done INT DEFAULT 0;
    DECLARE v_product_name VARCHAR(100);

    DECLARE cur_related CURSOR FOR
        SELECT oi2.Product_ID, COUNT(*) AS occurrence_count
        FROM ORDER_ITEM oi1
        JOIN ORDER_ITEM oi2 ON oi1.Order_ID = oi2.Order_ID
```

```
        WHERE oi1.Product_ID = p_product_id
          AND oi2.Product_ID != p_product_id
        GROUP BY oi2.Product_ID
        ORDER BY occurrence_count DESC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    -- Kiem tra dau vao
    IF p_product_id IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'LOI: Ma san pham khong duoc NULL';
    END IF;

    SELECT Name INTO v_product_name FROM PRODUCT WHERE Product_ID = p_product_id;
    IF v_product_name IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'LOI: San pham khong ton tai';
    END IF;

    -- Tinh tong so don hang co chua san pham chinh
    SELECT COUNT(DISTINCT Order_ID) INTO v_total_orders
    FROM ORDER_ITEM WHERE Product_ID = p_product_id;

    IF v_total_orders = 0 THEN
        RETURN CONCAT('San pham "', v_product_name, '" chua co lich su giao dich.');
    END IF;

    OPEN cur_related;
    product_loop: LOOP
        FETCH cur_related INTO v_related_product_id, v_occurrence_count;
        IF done THEN LEAVE product_loop; END IF;

        SET v_confidence = (v_occurrence_count * 100.0) / v_total_orders;

        IF v_confidence >= p_min_confidence THEN
            SELECT Name INTO v_related_product_name FROM PRODUCT
            WHERE Product_ID = v_related_product_id;

```

```
59              IF v_result = '' THEN
60                  SET v_result = CONCAT('- ', v_related_product_name, ' (', ROUND(
    v_confidence, 1), '%)');
61              ELSE
62                  SET v_result = CONCAT(v_result, '\n', '- ', v_related_product_name, ' ('
    , ROUND(v_confidence, 1), '%)');
63              END IF;
64              SET v_count = v_count + 1;
65          END IF;
66      END LOOP;
67      CLOSE cur_related;
68
69      IF v_count = 0 THEN
70          RETURN CONCAT('Khong co san pham goi y nao dat do tin cay >= ', ROUND(
    p_min_confidence, 1), '%');
71      END IF;
72
73      RETURN v_result;
74  END //
75  DELIMITER ;
```

Listing 13: Ham goi y san pham

2.4.2 Hàm phân hạng khách hàng (fn_get_customer_tier)

- Mục đích Hàm `fn_get_customer_tier` phân hạng khách hàng (BRONZE, SILVER, GOLD, DIAMOND) dựa trên tổng chi tiêu từ các đơn hàng đã hoàn thành.

- Input và Output

- **Input**: `p_customer_id INT` – Mã khách hàng.
- **Output**: `VARCHAR(50)` – Hạng thành viên (BRONZE/SILVER/GOLD/DIAMOND).

- Cách phân hạng

- **DIAMOND**: Tổng chi tiêu $\geq$ 10.000.000 VNĐ
- **GOLD**: 5.000.000 $\leq$ Tổng chi tiêu $<$ 10.000.000 VNĐ

- **SILVER**: 1.000.000 $\leq$ Tổng chi tiêu $<$ 5.000.000 VNĐ
- **BRONZE**: Tổng chi tiêu $<$ 1.000.000 VNĐ

- Logic xử lý

1. Kiểm tra tham số đầu vào và sự tồn tại của khách hàng.
2. Sử dụng `CURSOR` lấy tất cả các khoản thanh toán của khách hàng từ các đơn hàng `Completed`.
3. Tính tổng chi tiêu bằng vòng lặp.
4. Phân hạng dựa trên tổng chi tiêu.

- Demo

```
SELECT User_name, fn_get_customer_tier(12) AS Membership_Status
FROM USER WHERE User_ID = 12;
```

User_name	Membership_Status
oanh_buy	DIAMOND

Hình 2.16: Kết quả phân hạng khách hàng

- Mã nguồn

```
DROP FUNCTION IF EXISTS fn_get_customer_tier;
DELIMITER //

CREATE FUNCTION fn_get_customer_tier(p_customer_id INT)
RETURNS VARCHAR(50)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_order_total DECIMAL(15,2);
    DECLARE v_sum_spent DECIMAL(15,2) DEFAULT 0;
    DECLARE v_tier VARCHAR(20);
```

```
13
14      DECLARE cur_orders CURSOR FOR
15          SELECT Amount
16          FROM PAYMENT p
17          JOIN `ORDER` o ON p.Order_ID = o.Order_ID
18          WHERE o.Customer_ID = p_customer_id AND o.Order_status = 'Completed';
19
20      DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
21
22      -- Kiem tra dau vao
23      IF p_customer_id IS NULL THEN
24          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Loi: ID khach hang khong duoc de
    trong';
25      END IF;
26
27      IF NOT EXISTS (SELECT 1 FROM CUSTOMER WHERE User_ID = p_customer_id) THEN
28          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Loi: Khach hang khong ton tai';
29      END IF;
30
31      OPEN cur_orders;
32      order_loop: LOOP
33          FETCH cur_orders INTO v_order_total;
34          IF done THEN LEAVE order_loop; END IF;
35          SET v_sum_spent = v_sum_spent + v_order_total;
36      END LOOP;
37      CLOSE cur_orders;
38
39      -- Phan hang
40      IF v_sum_spent >= 10000000 THEN
41          SET v_tier = 'DIAMOND';
42      ELSEIF v_sum_spent >= 5000000 THEN
43          SET v_tier = 'GOLD';
44      ELSEIF v_sum_spent >= 1000000 THEN
45          SET v_tier = 'SILVER';
46      ELSE
47          SET v_tier = 'BRONZE';
```

```
    END IF;

    RETURN v_tier;
END //
DELIMITER ;
```

- Đánh giá Trong các test case đã chuẩn bị, hai hàm trả về kết quả đúng với dữ liệu mẫu và đáp ứng các yêu cầu nghiệp vụ:

- Hàm gợi ý sản phẩm giúp tăng trải nghiệm mua sắm và hỗ trợ bán chéo (cross-selling).
- Hàm phân hạng khách hàng hỗ trợ chiến lược chăm sóc khách hàng và tiếp thị cá nhân hóa.

3 Hiện thực ứng dụng

Ứng dụng OmniLogi được hiện thực theo mô hình web gồm ba lớp: frontend React, backend Node.js/Express và cơ sở dữ liệu MySQL. Frontend không truy cập trực tiếp vào MySQL mà gửi HTTP request đến backend. Backend nhận request, gọi stored procedure/function trong database, sau đó trả dữ liệu JSON về frontend.

- **Database**: MySQL 8 chạy bằng Docker, cổng trong container là `3306`, cổng máy host là `3307`.
- **Backend**: Node.js/Express, cổng API là `http://localhost:8080`.
- **Frontend**: React + Vite, khi chạy development server mặc định ở `http://localhost:5173`.
- **Cổng dữ liệu backend đưa ra**: toàn bộ dữ liệu ứng dụng được xuất qua các endpoint bắt đầu bằng `/api`, định dạng trả về là JSON.

3.1 Kiến trúc kết nối ứng dụng với CSDL

3.1.1 Kết nối database

Backend sử dụng thư viện `mysql2/promise` để tạo connection pool đến MySQL. Thông tin kết nối được đặt trong file `backend/.env`; database được map qua Docker ở cổng `3307`.

```
import dotenv from 'dotenv';
dotenv.config();
import mysql from 'mysql2/promise';

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

```

```
export { pool };
```

Listing 14: Tao connection pool den MySQL trong backend/config/database.js

Hàm `mysql.createPool` giúp tái sử dụng kết nối thay vì tạo kết nối mới cho từng request. Điều này phù hợp với ứng dụng web vì nhiều người dùng có thể gọi API cùng lúc. Các biến `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` giúp tách cấu hình ra khỏi code.

3.1.2 Cấu hình server và route

```
const app = express();
app.use(cors());
app.use(express.json());

app.use('/api', routes);
app.use(errorHandler);

const port = process.env.PORT || 8888;
app.listen(port, () => {
  console.log(`Example app listening at http://localhost:${port}`);
});
```

Listing 15: Cau hinh Express app va cong chay server

Express dùng `cors()` để frontend ở cổng khác có thể gọi API, dùng `express.json()` để đọc dữ liệu JSON từ request body. Biến môi trường `PORT = 8080` làm backend chạy tại `http://localhost:8080`. Các route nghiệp vụ được gom dưới tiền tố `/api`.

```
router.use('/products', productRoutes);
router.use('/stores', storeRoutes);
router.use('/customers', customerRoutes);
```

Listing 16: Cac nhom endpoint chinh trong backend/routes/index.js

Các endpoint chính của ứng dụng:

- `GET /api/products`: lấy danh sách sản phẩm từ thủ tục `sp_get_all_products`.
- `GET /api/products/:id`: lấy chi tiết sản phẩm.
- `POST /api/products`: thêm sản phẩm bằng `sp_insert_product`.

- `PUT /api/products/:id`: sửa sản phẩm bằng `sp_update_product`.
- `DELETE /api/products/:id`: xoá sản phẩm bằng `sp_delete_product`.
- `GET /api/products/:id/recommendations`: gọi hàm `fn_recommend_products`.
- `GET /api/customers/:id/tier`: gọi hàm `fn_get_customer_tier`.
- `GET /api/stores/revenue-stats`: gọi thủ tục `sp_get_store_revenue_stats`.

3.1.3 Định dạng dữ liệu trả về

Backend chuẩn hóa response bằng hai hàm `success` và `error`. Nhờ đó frontend luôn xử lý dữ liệu theo cùng một cấu trúc.

```
export const success = (res, data, message = 'OK') => {
    return res.json({
        success: true,
        data,
        message
    });
};

export const error = (res, message = 'Error', statusCode = 500) => {
    return res.status(statusCode).json({
        success: false,
        message
    });
}
```

Listing 17: Chuan hoa JSON response trong backend/utils/response.js

Ví dụ kết quả chạy API khi lấy danh sách sản phẩm:

```
{
  "success": true,
  "data": [
    {
      "Product_ID": 24,
      "Ten_San_Pham": "Sach lap trinh JavaScript",
      "Thuong_Hieu": "NXB Tre",
```

```
      "Danh_Muc": "Sach",
      "Trang_Thai": "Approved",
      "So_Cua_Hang_Ban": 1
    }
  ],
  "message": "Lay danh sach san pham thanh cong"
}
```

Listing 18: Ket qua JSON mau tu GET /api/products

3.2 Giao diện thêm, sửa, xoá dữ liệu vào bảng PRODUCT

Màn hình `ProductManager.jsx` đáp ứng yêu cầu 3.1 vì cho phép thêm, sửa, xoá dữ liệu trên bảng `PRODUCT`. Các thao tác này không ghi SQL trực tiếp ở frontend mà gọi API backend, sau đó backend gọi đúng ba thủ tục ở mục 2.1.

3.2.1 Thêm sản phẩm

Khi người dùng bấm nút `+ Thêm sản phẩm`, hàm `openAddModal` reset form về trạng thái tạo mới và mở popup nhập liệu. Khi bấm lưu, hàm `handleSave` gọi `POST /api/products`.

```
const openAddModal = () => {
    setEditingId(null);
    setForm({
        category_id: '1',
        name: '',
        desc: '',
        brand: '',
        image_url: '',
        status: 'Pending',
        created_by_id: ''
    });
    setIsModalOpen(true);
};

const insertPayload = {
    ...form,
    created_by_id: form.created_by_id ? parseInt(form.created_by_id) : null
};
```

```
await axiosClient.post('/products', insertPayload);
```

Listing 19: Gui yeu cau them san pham tu frontend

Ở backend, controller nhận request rồi chuyển dữ liệu xuống service:

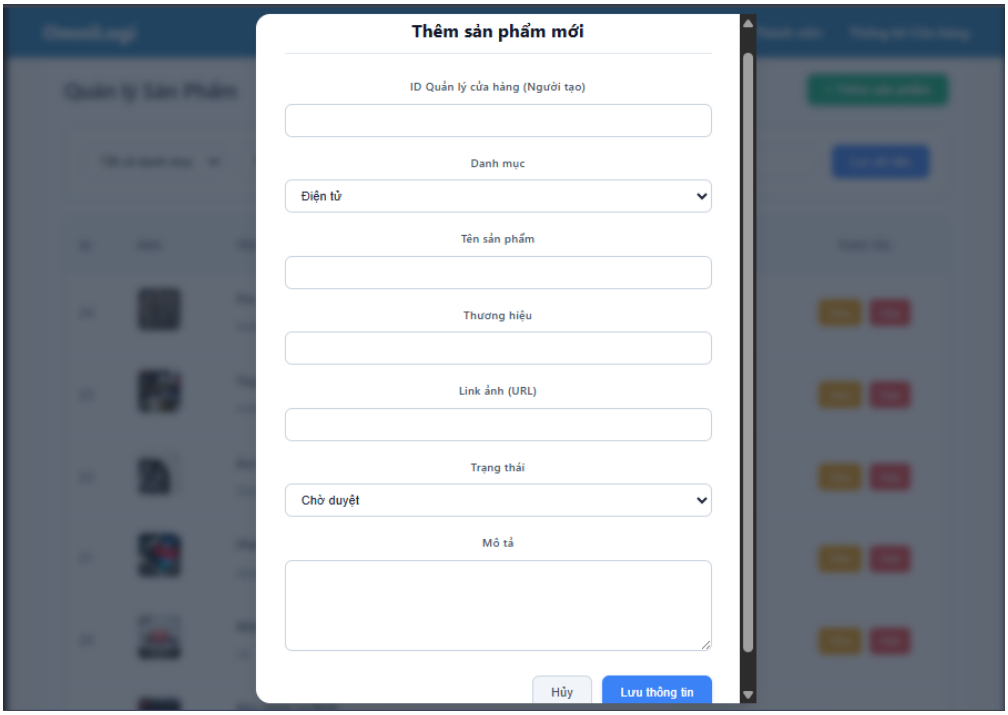
```
export const insertProduct = async (req, res, next) => {
    try {
        const data = await productService.insertProduct(req.body);
        return success(res, data, 'Them san pham thanh cong');
    } catch (err) {
        next(err);
    }
};
```

Listing 20: Controller them san pham

Service gọi thủ tục `sp_insert_product` trong MySQL:

```
const [rows] = await pool.query(
    `CALL sp_insert_product(?, ?, ?, ?, ?, ?, ?)`,
    [category_id, name, desc, brand, image_url, status, created_by_id]
);
```

Listing 21: Service goi thu tuc them san pham



Hình 3.1: Giao diện thêm sản phẩm mới

Kết quả chạy chức năng: nếu dữ liệu hợp lệ, frontend hiển thị thông báo `Thêm thành công!`, đóng popup và gọi lại `fetchProducts()` để tải danh sách mới. Nếu dữ liệu sai, backend trả lỗi từ stored procedure; frontend ưu tiên hiển thị nội dung `error.response.data.message` để người dùng biết lỗi cụ thể như sai danh mục, thiếu tên sản phẩm hoặc thiếu store manager.

3.2.2 Sửa sản phẩm

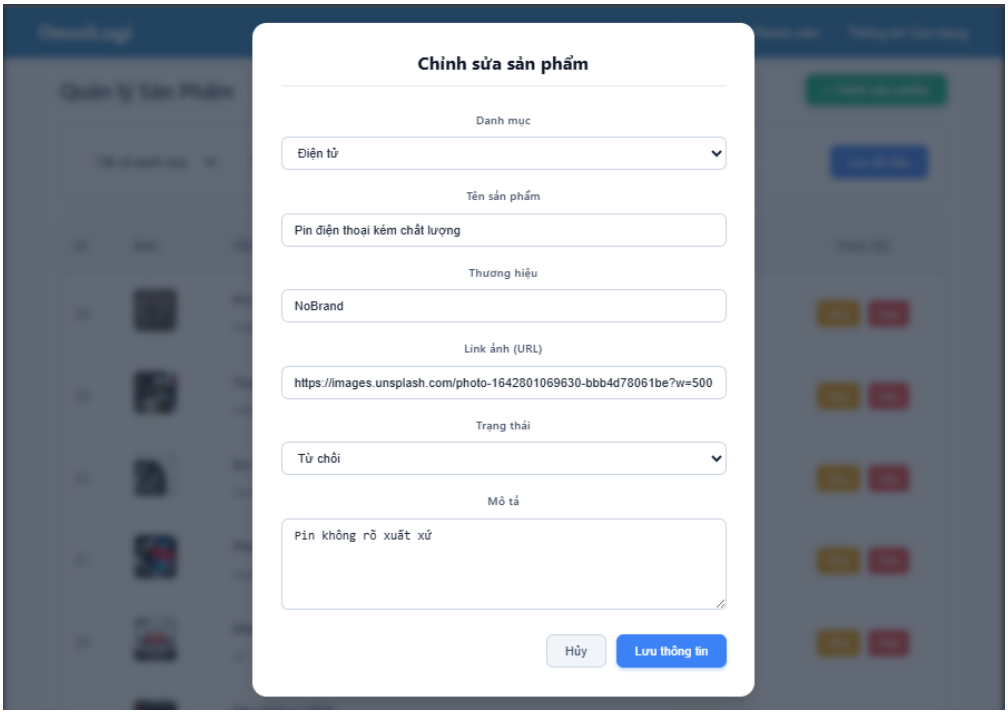
Khi người dùng chọn nút **Sửa**, frontend gọi `GET /api/products/:id` để lấy dữ liệu hiện tại của sản phẩm, đổ vào form rồi mở popup chỉnh sửa. Sau khi lưu, frontend gọi `PUT /api/products/:id`.

```
const openEditModal = async (id) => {
    const res = await axiosClient.get(`/products/${id}`);
    const p = res.data.data;
    setForm({
        category_id: p.Category_ID || '1',
        name: p.Name,
        desc: p.Description,
        brand: p.Brand,
        image_url: p.image_url,
```

```
        status: p.status,
        created_by_id: p.Created_by_ID || ''
    });
    setEditingId(id);
    setIsModalOpen(true);
};

await axiosClient.put(`/products/${editingId}`, updatePayload);
```

Listing 22: Mo popup sua va luu cap nhat san pham



Hình 3.2: Giao diện chỉnh sửa sản phẩm

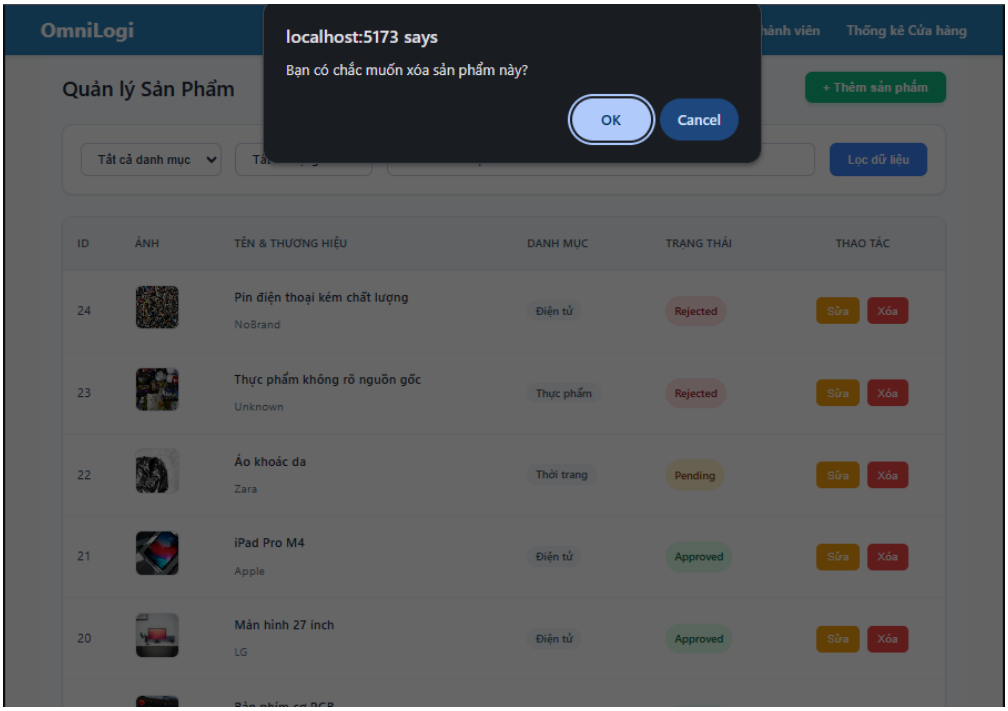
Kết quả chạy chức năng: nếu cập nhật hợp lệ, frontend thông báo `Cập nhật thành công!`. Nếu cố chuyển trạng thái một sản phẩm đã phát sinh đơn hàng khỏi `Approved`, backend trả lỗi nghiệp vụ từ `sp_update_product`.

3.2.3 Xoá sản phẩm

Chức năng xoá dùng hộp thoại xác nhận để tránh người dùng thao tác nhầm. Nếu xác nhận, frontend gọi `DELETE /api/products/:id`; backend tiếp tục gọi thủ tục `sp_delete_product`.

```
const handleDelete = async (id) => {
    if (!window.confirm("Ban co chac muon xoa san pham nay?")) return;
    try {
        await axiosClient.delete(`/products/${id}`);
        fetchProducts();
    } catch (error) {
        alert(error?.response?.data?.message || "Loi khi xoa san pham");
    }
};
```

Listing 23: Xoa san pham tu frontend



Hình 3.3: Giao diện xác nhận và kết quả xoá sản phẩm

Kết quả chạy chức năng: sản phẩm `Pending` hoặc `Rejected` chưa phát sinh nghiệp vụ được xoá thành công. Sản phẩm đã `Approved`, đã nằm trong giỏ hàng hoặc đã có đơn hàng sẽ bị chặn để bảo toàn lịch sử dữ liệu.

3.3 Giao diện danh sách, tìm kiếm và lọc sản phẩm

Màn hình danh sách sản phẩm xuất hiện ở cả `Home.jsx` và `ProductManager.jsx`. Đây là phần đáp ứng yêu cầu 3.2: hiển thị danh sách dữ liệu từ thủ tục 2.3, có tìm kiếm/filter, có thêm/sửa/xoá từ danh sách và xử lý lỗi logic thông qua backend. Việc sắp xếp hiện được thực hiện ở tầng database bằng `ORDER BY p.Product_ID DESC`, nên sản phẩm mới nhất luôn xuất hiện trước; giao diện chưa tách riêng một combobox chọn kiểu sắp xếp.

3.3.1 Gọi thủ tục `sp_get_all_products` từ giao diện

```
const fetchProducts = async () => {
    const res = await axiosClient.get('/products', {
        params: {
            status: statusFilter === 'All' ? null : statusFilter,
            search: searchFilter || null,
            category_id: categoryIdFilter || null
        }
    });
    setProducts(res.data.data || []);
};
```

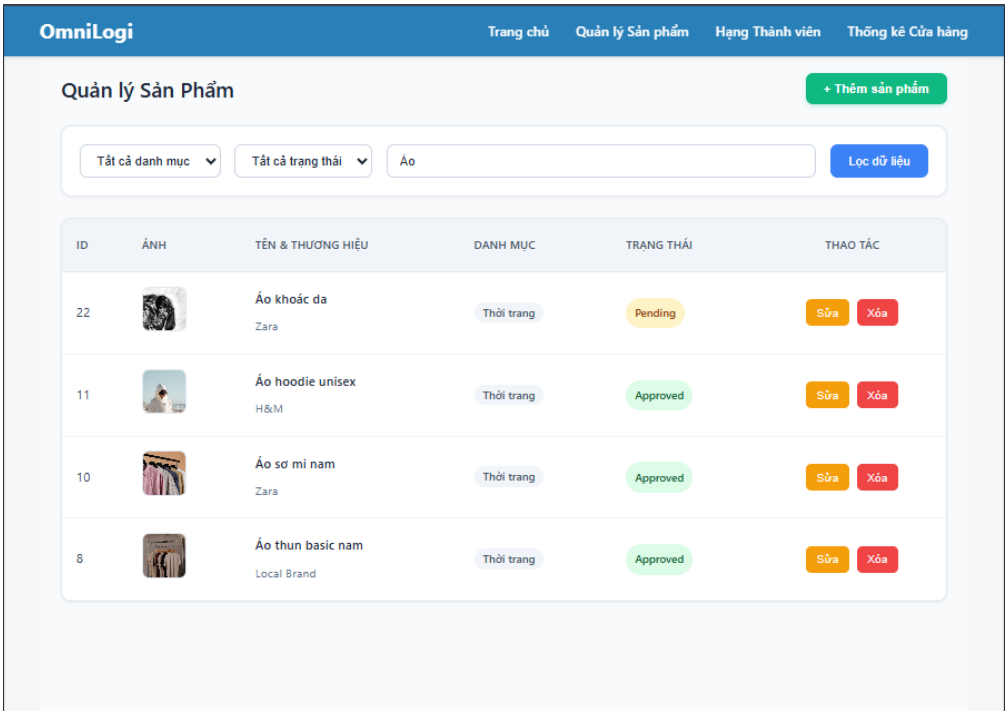
Listing 24: Frontend goi API danh sach san pham voi tham so filter

Ba tham số `status`, `search`, `category_id` được truyền qua query string. Backend nhận các tham số này và gọi thủ tục:

```
const [rows] = await pool.query(
    `CALL sp_get_all_products(?, ?, ?)`,
    [status, category_id, search]
);

return rows[0];
```

Listing 25: Backend goi sp_get_all_products



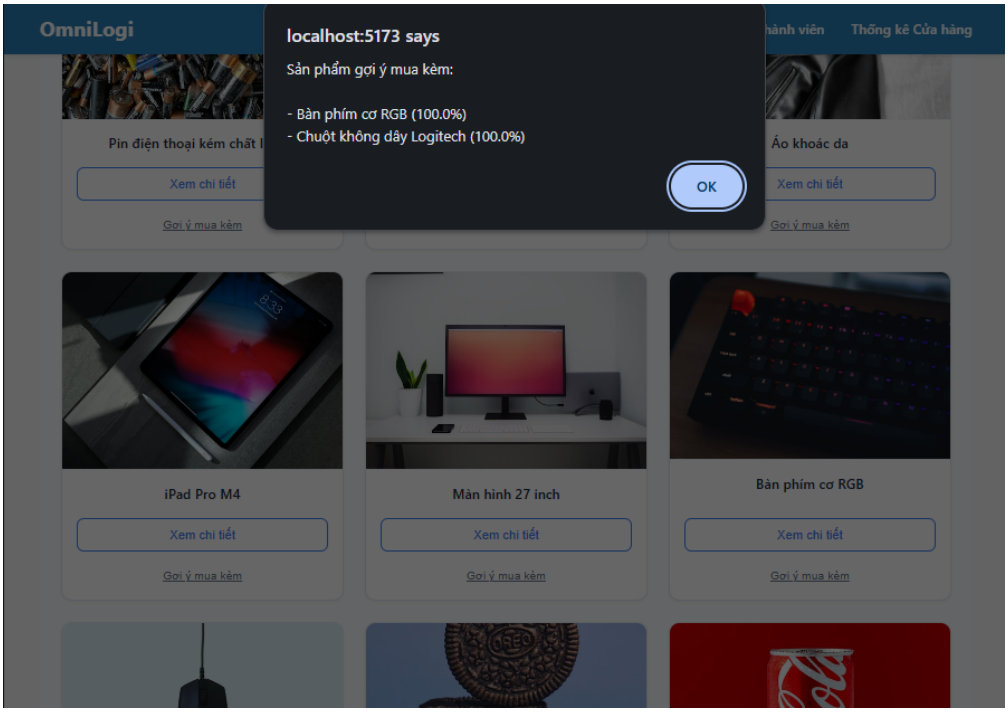
Hình 3.4: Giao diện tìm kiếm và lọc sản phẩm theo danh mục, trạng thái, từ khóa

3.3.2 Hiển thị chi tiết và gợi ý mua kèm

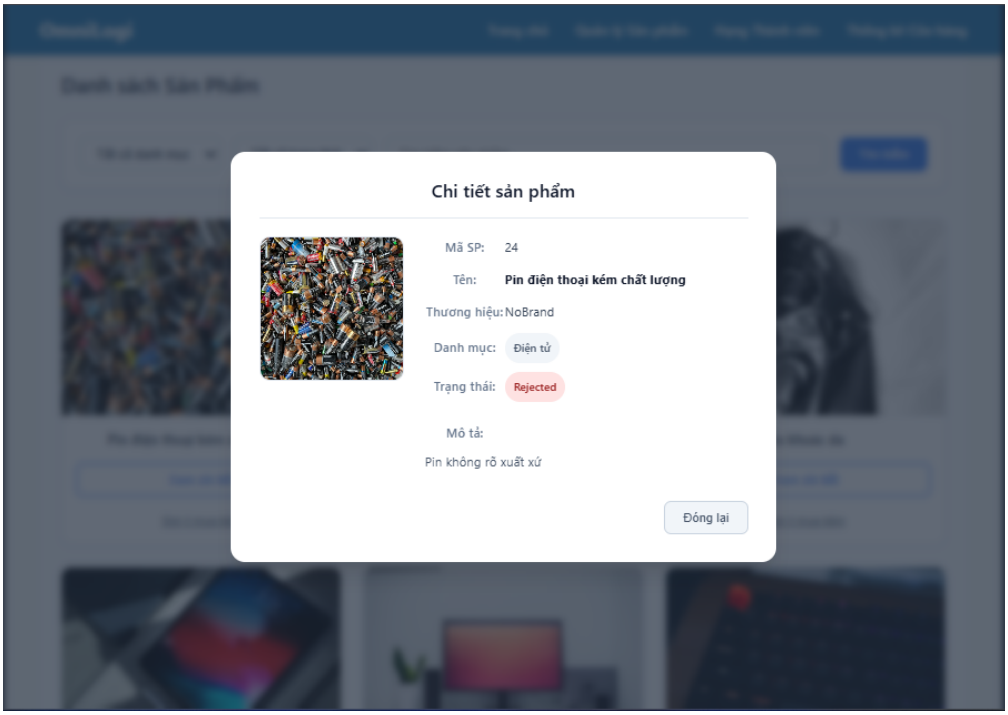
Ở trang chủ, mỗi sản phẩm có nút xem chi tiết và nút gợi ý mua kèm. Chức năng gợi ý gọi hàm `fn_recommend_products` thông qua endpoint `GET /api/products/:id/recommendations`.

```
const showRecommendations = async (id, e) => {
    e.stopPropagation();
    try {
        const res = await axiosClient.get(`/products/${id}/recommendations`);
        alert(`San pham goi y mua kem:\n\n${res.data.data.recommendations || res.data.
    data}`);
    } catch (error) {
        alert("Chua co goi y nao cho san pham nay.");
    }
};
```

Listing 26: Frontend goi ham goi y san pham



Hình 3.5: Giao diện chức năng gợi ý mua kèm sản phẩm



Hình 3.6: Popup xem chi tiết sản phẩm

3.4 Giao diện minh họa hàm và thủ tục khác

3.4.1 Giao diện phân hạng khách hàng

Màn hình `CustomerTier.jsx` minh họa hàm `fn_get_customer_tier`. Người dùng có thể tra cứu theo ID; phần tìm theo username dùng một danh sách autocomplete cục bộ lấy theo các customer trong dữ liệu mẫu để quy đổi username sang `User_ID` trước khi gọi API. Frontend gọi endpoint `GET /api/customers/:id/tier`; backend trả về hạng `BRONZE`, `SILVER`, `GOLD` hoặc `DIAMOND`.

```
const checkTier = async () => {
    const idToCheck = searchType === 'id' ? inputValue : selectedId;
    if (!idToCheck) {
        alert("Vui long nhap hoac chon khach hang hop le!");
        return;
    }

    const res = await axiosClient.get(`/customers/${idToCheck}/tier`);
    setTierData(res.data.data);
```

```
};
```

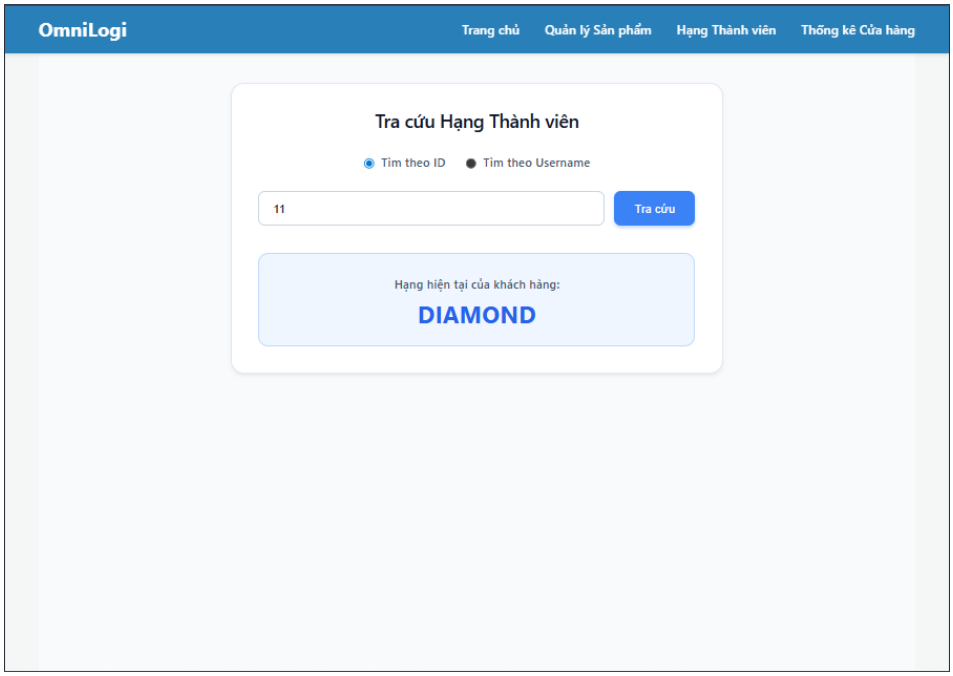
Listing 27: Frontend tra cuu hang khach hang

Backend gọi hàm trong database như sau:

```
const [rows] = await pool.query(
    `SELECT fn_get_customer_tier(?) AS tier`,
    [customerId]
);

return rows[0];
```

Listing 28: Backend goi fn_get_customer_tier



Hình 3.7: Giao diện tra cứu và hiển thị hạng thành viên khách hàng

Kết quả chạy chức năng: ví dụ khi tra cứu khách hàng có `User_ID = 12`, backend trả JSON dạng:

```
{
  "success": true,
  "data": {
```

```
    "tier": "DIAMOND"
  },
  "message": "Lay hang khach hang thanh cong"
}
```

Listing 29: Ket qua JSON mau tu GET /api/customers/12/tier

3.4.2 Giao diện thống kê doanh thu cửa hàng

Màn hình `StoreRevenue.jsx` minh họa thủ tục `sp_get_store_revenue_stats`. Người dùng có thể lọc theo mã cửa hàng, ngày bắt đầu, ngày kết thúc và doanh thu tối thiểu.

```
const fetchStats = async () => {
    const res = await axiosClient.get('/stores/revenue-stats', {
        params: {
            store_id: filter.store_id || null,
            from_date: filter.from_date || null,
            to_date: filter.to_date || null,
            min_revenue: filter.min_revenue || null
        }
    });
    setStats(res.data.data || []);
};
```

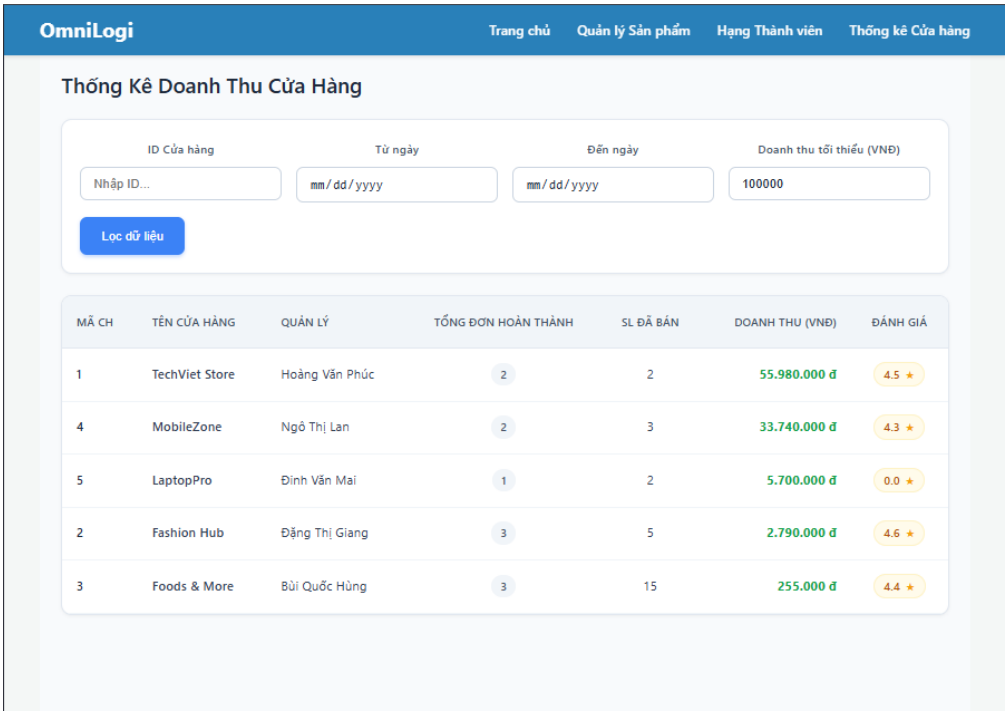
Listing 30: Frontend goi API thong ke doanh thu cua hang

Backend service gọi stored procedure:

```
const [rows] = await pool.query(
    `CALL sp_get_store_revenue_stats(?, ?, ?, ?)`,
    [store_id, from_date, to_date, min_revenue]
);

return rows[0];
```

Listing 31: Backend goi sp_get_store_revenue_stats



Hình 3.8: Giao diện thống kê doanh thu cửa hàng theo bộ lọc

Kết quả chạy chức năng: bảng thống kê hiển thị mã cửa hàng, tên cửa hàng, quản lý, số đơn hoàn thành, số lượng đã bán, doanh thu và điểm đánh giá trung bình. Dữ liệu được sắp xếp giảm dần theo doanh thu ở stored procedure, phù hợp với `ORDER BY` trong mục 2.3.

3.5 Đánh giá mức độ đáp ứng yêu cầu

- Ứng dụng kết nối thật với CSDL MySQL thông qua backend Express và connection pool.
- Giao diện CRUD sản phẩm gọi đúng ba thủ tục `sp_insert_product`, `sp_update_product`, `sp_delete_product`.
- Giao diện danh sách sản phẩm gọi thủ tục `sp_get_all_products`, có tìm kiếm, lọc danh mục, lọc trạng thái và thao tác sửa/xoá từ danh sách.
- Giao diện phân hạng khách hàng gọi hàm `fn_get_customer_tier`.
- Giao diện gợi ý mua kèm gọi hàm `fn_recommend_products`.
- Giao diện thống kê doanh thu gọi thủ tục `sp_get_store_revenue_stats`.
- Backend xuất dữ liệu qua cổng 8080 ở định dạng JSON; database chạy qua cổng host 3307.

- Các lỗi nghiệp vụ quan trọng được xử lý trong stored procedure/function, trả về backend qua middleware `errorHandler`, sau đó frontend hiển thị nội dung lỗi cụ thể nếu response có trường `message`.

3.6 Source code

Source code GitHub các phần:

- **SQL scripts**: https://github.com/tiennd2k5/Omnilogi/tree/main/database
- **Frontend**: https://github.com/tiennd2k5/Omnilogi/tree/main/frontend
- **Backend**: https://github.com/tiennd2k5/Omnilogi/tree/main/backend

Bibliography

[1] Shopee, *Nền tảng thương mại điện tử*. Truy cập tại: https://shopee.vn

[2] Ramez Elmasri, Shamkant B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016. Truy cập tại: https://library.uc.edu.kh/userfiles/pdf/7.Fundamentals%20of%20Database%20Systems.pdf

[3] GeeksforGeeks, *Mapping from ER Model to Relational Model*. Truy cập tại: https://www.geeksforgeeks.org/dbms/mapping-from-er-model-to-relational-model

[4] Draw.io (diagrams.net), *Công cụ trực tuyến vẽ sơ đồ EERD và mô hình dữ liệu*. Truy cập tại: https://app.diagrams.net